



Invited Review in Celebration of the 50th Anniversary of EURO: Forty Years of Interior Point Methods

Interior point methods in the year 2025

Jacek Gondzio ^{ID}

School of Mathematics and Maxwell Institute for Mathematical Sciences, The University of Edinburgh, Peter Guthrie Tait Road, Edinburgh EH9 3FD, United Kingdom

ARTICLE INFO

Keywords:

Interior point methods
Linear programming
Conic programming
Theory and computational perspective
History and future developments

ABSTRACT

Interior point methods (IPMs) have hugely influenced the field of optimization. Their fast development has been triggered by the seminal paper of Narendra Karmarkar published in 1984 which delivered a polynomial algorithm for linear programming and suggested that it might be implemented into a very efficient method in practice. Indeed, this has been demonstrated within a few years after 1984 and has gained IPMs a status of exceptionally powerful optimization tool. Linear Programming (LP) is at the centre of many operational research techniques including mixed-integer programming, network optimization and various decomposition techniques. Therefore, any progress in LP has far-reaching consequences. IPMs certainly did not disappoint in this context: they have become a heavily used methodology in modern optimization and operational research. Their accuracy, efficiency and reliability have been particularly appreciated when IPMs are applied to truly large scale problems which challenge any alternative approaches. In this survey we will discuss several issues related to interior point methods. We will recall techniques which provide the building blocks of IPMs, and observe that actually at least some of them have been developed before 1984. We will briefly comment on the worst-case complexity results for different variants of IPMs and then focus on key aspects of their implementation. We will also address some of the most spectacular features of IPMs and discuss their potential advantages when applied in decomposition algorithms, cutting planes scheme and column generation technique.

1. Introduction

Linear Programming (LP) is a key modelling and optimization technique and its use within the area of Operational Research (OR) cannot be underestimated. Since 1940s researchers have been developing various LP solution techniques and three of them, the simplex algorithm [81,35], ellipsoid method [86] and interior point algorithm [82] stand out for their practical or theoretical significance. Although their official dates of birth are 1947, 1979 and 1984, respectively, the ideas which they rely on can often be traced back years or decades earlier.

For at least four decades, the simplex algorithm was the only reliable solution technique for LP and this has had huge consequences for the development of other methods in OR and for the education of at least two generations of OR specialists. The need to go beyond the restrictive area of linear objectives and linear constraints have forced researchers to embed LPs and the simplex method into other more sophisticated algorithms. Integer programming has been using the simplex method for LP as a tool to solve continuous relaxations

E-mail address: J.Gondzio@ed.ac.uk.

URL: <http://maths.ed.ac.uk/~gondzio/>.

<https://doi.org/10.1016/j.ejco.2025.100105>

in a variety of schemes which involve branching, bounding and elaborated cut generation and pricing techniques [54,114]. Early attempts to solve nonlinear programming problems relied on local linearization of nonlinear constraints and solving a sequence of linear approximations of the problem [113]. The need to tackle large and otherwise intractable problems led to the development of various decomposition techniques [15,36].

The simplex method visits the vertices of the polytope defined by the linear constraints and moves along the edges of this polytope which corresponds to changing the activity of only a single inequality constraint. This feature allows for a very efficient implementation of the method which involves a local representation of active constraints and perfectly exploits the inherent sparsity of LP in the linear algebra operations. It makes the implementation of the simplex method very powerful, and indeed modern simplex solvers demonstrate impressive performance on many difficult LP instances. However, the simplex method has a theoretical drawback: it might have to visit many vertices of the feasible polytope before finding the optimal one. In the worst-case it might have to visit a non-polynomial number of vertices [87]. Many researchers have been looking for an illusive variant of the simplex (or simplex-like) method which would enjoy a polynomial worst-case complexity [133], but it is hard to believe that one will ever be found.

The ellipsoid algorithm, developed by Khachiyan [86] was the first method for LP with a polynomial worst-case complexity and a major breakthrough in the field of optimization. Unfortunately, the linear algebra operations involved in the update of ellipsoids cannot take advantage of the sparsity of the problem data and require a very significant computational effort. Moreover, the ellipsoid method seems to behave like in its worst case analysis and overall it is disappointingly slow [57].

It was Karmarkar's projective LP algorithm [82] which completely changed the optimization landscape in 1984. By combining projective geometry, balls inscribed into the feasible polytope and a cleverly designed potential function, Karmarkar was able to demonstrate that a constant reduction of the appropriately chosen merit function is achievable at each iteration and therefore the method had to reach an optimal solution in a polynomial number of iterations. It took a few years before the first implementations of the method could deliver efficient practical algorithms, but already in late 1980s the consensus was that for large problems IPMs would provide a viable alternative to the simplex method. It was soon accepted that the larger the problems, the relatively more computationally attractive IPMs become [97,98,102].

Although initially interior point methods were meant to solve LPs, they were relatively quickly extended to deal with quadratic and nonlinear problems and then also adopted to solve the second-order cone and semidefinite programming problems. Therefore, they have provided a unified framework to deal with various classes of optimization problems which involve inequalities defined by the use of convex functions [116]. Today, forty years after Karmarkar's paper was published, IPMs have an established position as the methods of choice for solving very large and difficult optimization problems.

In this paper we will introduce all key notions relevant for the development of interior point methods. We will focus on linear programming problems, but will also comment on IPM extensions to quadratic, nonlinear, second-order cone and semidefinite problems. While introducing IPMs, we will give a historical perspective on some of the major contributions made in the field of optimization which may have influenced their development. In Section 2 we will introduce the linear optimization problem, define the notation used in the paper and discuss the main mathematical building blocks of IPMs such as the use of logarithmic barrier functions, the role of the central path, the way in which IPMs satisfy the complementarity condition and key features responsible for IPMs' impressive polynomial convergence properties. Then in Section 3 we will present a practical algorithm and address the key differences between the two competing LP solution techniques: the simplex method and IPMs. In Section 4 we will provide a broader perspective and comment on the use of IPMs to solve other classes of optimization problems such as quadratic, nonlinear, second-order cone and semidefinite programming problems. We will introduce self-concordant barriers and comment on their importance in the polynomial worst-case complexity analysis. We will also mention some interesting recent theoretical developments and give the author's personal view of the possible future developments in the area. In Section 5 we will return to the discussion of practical aspects related to the implementation of modern IPMs. We will focus on the use of iterative schemes as an alternative to direct methods for the related linear algebra problems. We will also comment on the importance and challenges of designing preconditioners for iterative methods. In Section 6 we will address advantages of employing interior point methods in the context of decomposition algorithms, cutting plane method and column generation scheme. We will illustrate it with one particular example, a discrete optimal transport problem. Finally, in Section 7 we will draw our conclusions.

2. Interior point methods: background

In this section we will introduce linear programming problems and perform basic steps to derive an interior point method for their solution. While doing so we will comment on key features of IPM methodology and summarise the current state-of-the-art in theory of IPMs. We will also provide a historical perspective on some of the related developments which may have contributed to a better understanding of various building blocks of IPMs but might have been missed by the mainstream literature.

We consider a primal-dual pair of linear programming problems

Primal	Dual	
$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \\ & x \geq 0; \end{aligned}$	$\begin{aligned} \max \quad & b^T y \\ \text{s.t.} \quad & A^T y + s = c, \\ & y \text{ free}, \quad s \geq 0, \end{aligned}$	(1)

where $A \in \mathcal{R}^{m \times n}$ has full row rank $m \leq n$, $x, s, c \in \mathcal{R}^n$ and $y, b \in \mathcal{R}^m$. The first order optimality conditions for these problems follow easily from the Lagrangian duality theory [20] and give:

$$\begin{aligned}
Ax &= b, \\
A^T y + s &= c, \\
XSe &= 0, \\
x, s &\geq 0.
\end{aligned} \tag{2}$$

We have used here a notation which is common for the IPM literature [144] and write the complementarity condition in the form of the equation $XSe = 0$, where X and S are diagonal matrices in $\mathcal{R}^{n \times n}$ with elements of the vectors x and s spread across the diagonal, respectively, and $e \in \mathcal{R}^n$ is the vector of ones. By unfolding the matrix notation, this condition may be equivalently written down in a component-wise form as $x_j s_j = 0, \forall j \in \{1, 2, \dots, n\}$.

The complementarity condition is a consequence of the presence of inequalities in the primal and dual problems: $x \geq 0$ and $s \geq 0$, respectively. Inequalities are a source of difficulty in optimization. Unlike equality constraints, any *inequality* $x_j \geq 0$ may be satisfied at the optimal solution either as the *inactive* constraint ($x_j > 0$) or as the *active* one ($x_j = 0$). In case of linear programming, knowing the activity of the constraints is (almost) equivalent to knowing an optimal solution of the problem.

The two competing solution methods for linear programming (the simplex method and IPMs) differ essentially in their ways of dealing with the complementarity condition. The simplex method forces the complementarity condition to be satisfied at each iteration. It achieves the goal by partitioning the set of indices $\{1, 2, \dots, n\}$ into two disjoint subsets $\mathcal{B}$ and $\mathcal{N}$. For all basic indices $j \in \mathcal{B}$ it forces $s_j = 0$ and allows x_j to take a nonzero value, while for all nonbasic indices $j \in \mathcal{N}$ it forces $x_j = 0$ and allows s_j to take a nonzero value. IPMs keep all complementarity products $x_j s_j$ strictly positive at each iteration, but systematically (and uniformly) reduce them until they get close enough to zero. Close to optimality one of the variables x_j and s_j is expected to be significantly smaller than the other, thus delivering an initial guess of an optimal partition into “basic” and “nonbasic” indices. The process of identifying an optimal basis from a nearly optimal strongly complementary solution obtained by an IPM is not trivial. Theoretical aspects of it have been addressed in [101]. Practical implementation of this step, called crossover, involves simplex-type updates and relies on the ability to identify stable basis matrices. The use of maximum volume basis (see [127,128]) delivers computational advantages in this context.

IPMs use the logarithmic barrier function $f : \mathcal{R} \rightarrow \mathcal{R}$

$$f(x) = \begin{cases} -\ln(x) & \text{if } x > 0 \\ +\infty & \text{otherwise,} \end{cases} \tag{3}$$

to “replace” every nonnegativity constraint $\{x \in \mathcal{R}, x \geq 0\}$. The original primal problem (1) is then replaced with the primal barrier LP problem

$$\min c^T x - \mu \sum_{j=1}^n \ln x_j \quad \text{s.t.} \quad Ax = b, \tag{4}$$

in which the parameter μ controls the relation between the force of the barrier term and that of the original LP objective. A very large value of μ makes the LP objective negligible and forces the choice of a point which satisfies equality constraints and minimizes $-\sum_{j=1}^n \ln x_j$ hence it maximizes $\prod_{j=1}^n x_j$ and therefore prevents any x_j from approaching zero. On the other hand, any very small value of μ in (4) underlines the relative importance of term $c^T x$ in the objective while still “keeping” all variables x_j away from zero. The transformation which employs logarithmic barriers to handle nonnegativity constraints removes explicit inequalities, but any IPM algorithm which uses it has to (implicitly) maintain the strict positivity of all variables x_j .

With the equality constrained problem (4) we associate the Lagrangian

$$\mathcal{L}(x, y) = c^T x - \mu \sum_{j=1}^n \ln x_j - y^T (Ax - b) \tag{5}$$

and write the first order optimality conditions

$$\begin{aligned}
\nabla_y \mathcal{L}(x, y) &= Ax - b = 0, \\
\nabla_x \mathcal{L}(x, y) &= c - A^T y - \mu X^{-1} e = 0.
\end{aligned} \tag{6}$$

By introducing a new variable $s = \mu X^{-1} e$ (i.e. $XSe = \mu e$) the first order optimality conditions become

$$\begin{aligned}
Ax &= b, \\
A^T y + s &= c, \\
XSe &= \mu e, \\
x, s &\geq 0.
\end{aligned} \tag{7}$$

It is easy to observe that any point (x, y, s) which satisfies these conditions is both primal and dual feasible for (1). The corresponding duality gap is equal to the complementarity gap

$$c^T x - b^T y = (A^T y + s)^T x - b^T y = y^T (Ax) + s^T x - b^T y = x^T s = n\mu, \tag{8}$$

hence the barrier parameter μ controls the distance to optimality.

A comparison of (2) and (7) reveals an essential difference between the mechanism used by the simplex method and the interior point method to approach an optimal solution. The (feasible) IPM attempts to keep all complementarity products $x_j s_j \approx \mu$, and forces

convergence by gradually reducing the barrier term μ . It is worth looking once again at the barrier problem (4) and realising that the reduction of μ means that, as the relative importance of term $c^T x$ increases, the *centrality* of the point in the feasible polytope induced by $\mu \sum_{j=1}^n \ln x_j$ is traded for more and more of *optimality*. It is worth mentioning at this point that the important role of centres in optimization algorithms predates the development of IPMs [25,76].

IPMs replace problem (1) with a sequence of barrier problems (4) using a suitable decreasing sequence of barrier parameters $\{\mu^k\}$. There is no need to solve any given barrier subproblem to optimality. Instead, a single Newton step is made for such a problem and the barrier term is updated (reduced) from μ^k to $\mu^{k+1} = \sigma \mu^k$ with a suitable $\sigma \in (0, 1)$.

Newton's method is applied to the system of mildly nonlinear first-order optimality conditions (7). The Newton direction $(\Delta x, \Delta y, \Delta s)$ is computed by solving the following sparse and highly structured system of linear equations

$$\begin{bmatrix} A & 0 & 0 \\ 0 & A^T & I_n \\ S & 0 & X \end{bmatrix} \cdot \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta s \end{bmatrix} = \begin{bmatrix} \xi_p \\ \xi_d \\ \xi_\mu \end{bmatrix} = \begin{bmatrix} b - Ax \\ c - A^T y - s \\ \sigma \mu e - XSe \end{bmatrix}, \quad (9)$$

where I_n denotes the identity matrix of dimension n . It is worth observing that the right-hand side of the third block equation involves a target barrier term $\sigma \mu$, which means that the Newton direction points towards a solution of (7) which guarantees progress in reducing the complementarity gap.

The solution of Newton system (9) constitutes the main computational effort in IPMs. An implementation of the linear algebra technique to perform this task determines the practical efficiency of the method. The linear system (9) involves three diagonal matrices (X, S and I_n) which allows to perform an easy elimination of one or two of the unknowns. For example, from the third block equation we may derive $\Delta s = X^{-1}(\xi_\mu - S\Delta x)$ and substitute into the second block equation giving the following symmetric but indefinite augmented system

$$\begin{bmatrix} -\Theta^{-1} & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} f \\ g \end{bmatrix} = \begin{bmatrix} \xi_d - X^{-1}\xi_\mu \\ \xi_p \end{bmatrix}, \quad (10)$$

where $\Theta = XS^{-1}$ is the so-called IPM scaling matrix. Since Θ is diagonal, the elimination process may be continued and when $\Delta x = \Theta(A^T \Delta y - f)$ is substituted into the second equation in (10) the following symmetric and positive definite normal equations system is obtained

$$(A\Theta A^T)\Delta y = h = A\Theta f + g. \quad (11)$$

It is worth having a closer look again at the first-order optimality conditions (7) and the corresponding Newton system (9) because they are crucial to a theoretical analysis of any interior point method (for LP). System (7) has a unique solution $(x(\mu), y(\mu), s(\mu))$, $x(\mu) > 0$, $s(\mu) > 0$ for any value of $\mu > 0$. Such a solution is called a μ -centre and a family of these solutions for all $\mu \in (0, \infty)$ forms a continuous path called the *primal-dual central path* [100]. When μ approaches zero, the optimality conditions for the μ -barrier LP problem (4) become the first order optimality conditions (2) for the original LP. Hence the central path terminates at an optimal solution of LP (1). Most of today's IPMs attempt to follow the central path, some follow it very closely indeed.

The closeness to the central path is enforced by the requirement of all iterates to belong to an appropriately defined *neighbourhood* of the path and this in common parlance means that the iterates should satisfy certain *centrality* conditions. For a feasible primal-dual IPM, all iterates (x, y, s) belong to $\mathcal{F}^0 = \{(x, y, s) \mid Ax = b, A^T y + s = c, (x, s) > 0\}$, that is, they satisfy the first two equations in (7). Additionally they approximately satisfy the third one: $XSe \approx \mu e$. The feature of staying close to the central path explains an alternative name of IPMs as *path-following methods* [69,100].

The parameter $\sigma \in (0, 1)$ in the third equation in (7) determines the aspiration of reducing the complementarity (and duality) gap.

In the *short step method*, $\sigma = 1 - \mathcal{O}(1)/\sqrt{n}$ is very close to 1 and consistently with it all iterates are restricted to stay in a tiny neighbourhood of the central path:

$$N_2(\theta) = \{(x, y, s) \in \mathcal{F}^0 \mid \|XSe - \mu e\|_2 \leq \theta \mu\}, \quad (12)$$

where $\theta \in (0, 1)$ is an arbitrary parameter and the barrier term is equal to the average complementarity product $\mu = x^T s / n$ (see (8)). Observe that when μ approaches zero and the central path approaches an optimal solution the size of the neighbourhood is also decreasing.

In the *long step method*, much more aggressive reduction of the duality gap is sought by setting σ to a small fraction, say, $\sigma \in (0.01, 0.5)$, and using wider neighbourhoods of the central path: either the infinity neighbourhood

$$N_\infty(\gamma) = \{(x, y, s) \in \mathcal{F}^0 \mid x_j s_j \geq \gamma \mu, \forall j\}, \quad (13)$$

or a symmetric neighbourhood

$$N_S(\gamma) = \{(x, y, s) \in \mathcal{F}^0 \mid \gamma \mu \leq x_j s_j \leq \frac{1}{\gamma} \mu, \forall j\}, \quad (14)$$

where $\gamma \in (0, 1)$. Both $N_\infty(\gamma)$ and $N_S(\gamma)$ leave significantly more room for the algorithm than $N_2(\theta)$ and allow for much larger progress to be made at each iteration. It is somewhat confusing that the steps taken in Newton directions are large in the short-step methods in which full steps are made while in the long step methods usually only damped steps are made. The short-step/long-step classification is based on "steps to optimality" and not on stepsizes in the Newton direction.

In practice there is no need to force feasibility of interior point iterates [88,89,152]. Since the first two equations in (7) which state the primal and dual feasibility are linear, the Newton method tends to satisfy them easily. Indeed, the most efficient implementations of IPMs are *infeasible* algorithms, the name of which originates from [152]. These methods attain feasibility along with reducing the complementarity (and duality) gap. Infeasible primal-dual path following methods use much wider neighbourhoods of the central path because they do not restrict (x, y, s) to stay in $\mathcal{F}^0$.

The interpretation of the third equation in (9) is crucial for the understanding of IPMs for LP. Observe that setting $\sigma = 1$ corresponds to pure recentring, i.e., attempting to get to the exact μ -centre (7). On the other hand, setting $\sigma = 0$ corresponds to pure optimization, i.e., attempting to get to the solution of problem (1) in one step, which usually is a hugely over-ambitious goal. The Newton direction which corresponds to $\sigma = 0$ is called the affine-scaling direction. An interior point-like method which makes steps in affine-scaling directions was proposed by Dikin [42] as early as in 1967. However, the global convergence of the affine-scaling algorithm was only established using the IPM perspective in the early 1990s [135]. To guarantee convergence, an affine-scaling algorithm has to take cautious steps in Newton directions (their stepsizes should not exceed 2/3), and this bound was shown to be sharp [75]. There is no proof of polynomial complexity of the affine-scaling algorithm. The presence of the “centrality” component in the direction seems to be crucial to be able to deliver it. The reader interested in a primal-dual affine-scaling algorithm is referred to [111].

The success of IPMs originates from combining the optimization feature of the affine-scaling direction with the recentring controlled by $\sigma \in (0, 1)$ which ensures that the iterates remain close to the central path. The third equation in the Newton system (9) has the following form:

$$S\Delta x + X\Delta s = \sigma\mu e - XSe.$$

Summing these equations (multiplying the above with e^T from the left) and using $XSe = \mu e$ gives

$$s^T\Delta x + x^T\Delta s = \sigma\mu e^T e - x^T s = (\sigma - 1)x^T s$$

and provides a handy expression for a linear estimate of the reduction of complementarity gap which may be achieved when a step along Newton’s direction is made. Indeed, after a step from the current point (x, y, s) in the direction $(\Delta x, \Delta y, \Delta s)$, with stepsize α ($\alpha \leq 1$), the complementarity gap at the new point $(\bar{x}, \bar{y}, \bar{s}) = (x, y, s) + \alpha(\Delta x, \Delta y, \Delta s)$ is

$$\begin{aligned}\bar{x}^T \bar{s} &= (x + \alpha\Delta x)^T (s + \alpha\Delta s) \\ &= x^T s + \alpha(s^T\Delta x + x^T\Delta s) + \alpha^2\Delta x^T\Delta s \\ &= (1 - \alpha(1 - \sigma))x^T s + \alpha^2\Delta x^T\Delta s.\end{aligned}\tag{15}$$

If the second order error term in the Newton method $\Delta x^T\Delta s$ is kept small, then by choosing α small enough the new gap $\bar{x}^T\bar{s}$ may be guaranteed to be smaller than the old one $x^T s$.

When the short-step feasible IPM is used, even more favourable conditions are met because $\Delta x^T\Delta s$ is zero and a full step $\alpha = 1$ in Newton’s direction may be taken. In such a case, assuming $\sigma = 1 - \beta/\sqrt{n}$ for some $\beta \in (0, 1)$, from (15) we get

$$\bar{x}^T \bar{s} = (1 - \alpha(1 - \sigma))x^T s + \alpha^2\Delta x^T\Delta s = (1 - \beta/\sqrt{n})x^T s.$$

This seemingly small per-iteration decrease of the complementarity gap is sufficient to guarantee a significant enough reduction after $\mathcal{O}(\sqrt{n})$ IPM iterations. Indeed, after $k = \sqrt{n}$ iterations the complementarity gap satisfies

$$\frac{\mu^k}{\mu^0} = (1 - \beta/\sqrt{n})^{\sqrt{n}} \approx e^{-\beta},$$

where e denotes Euler’s number $e \approx 2.71828$. After a number of iterations proportional to $\sqrt{n}$ the algorithm gets arbitrarily close to a solution. In the language of complexity theory the feasible primal-dual IPM converges to an ϵ -accurate solution in $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ iterations.

It is worth mentioning that significant efforts are being made to lower the theoretical computational complexity of IPMs for LP. Those include attempts to improve the iteration worst-case complexity [92] as well as reducing the computational complexity of a single iteration, see [34] and the references therein. Getting the iteration worst-case complexity below the current best bound of $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ seems elusive. This is somewhat disappointing because the observed average number of iterations to reach optimality is certainly well below this bound. There have been interesting attempts of involving new measures of conditioning in LP into the complexity analysis [123,141]. They provide a different perspective on this important theoretical problem.

Another relevant research question considers the existence of a strongly polynomial algorithm for linear programming. There were hopes that IPMs might be able to deliver such an algorithm. Shedding any light on this problem requires a better understanding of nuances concerning a possible behaviour of the central path [41]. Recent analyses of this problem [5,6], based on the use of tropical geometry, have provided a clarification that IPMs which involve logarithmic barriers (and more generally, the self-concordant barriers) and use the concept of following closely the central path cannot provide a strongly polynomial algorithm for LP.

3. Practical interior point methods

Although the short-step feasible primal-dual IPM delivers the best known guarantee of the worst-case complexity, it is a completely impractical algorithm. When implemented following the theory it would indeed converge in the number of iterations as suggested by its worst-case complexity. A significantly better *practical* algorithm can be produced when several over-restrictive assumptions needed by the short-step feasible primal-dual IPM are dropped, namely if:

- the requirement of feasibility of all iterates is relaxed;
- a wide central path neighbourhood is used: either $N_\infty(\gamma)$ or $N_S(\gamma)$ instead of $N_2(\theta)$; and
- the aggressive goal of reducing duality gap is defined (and damped Newton steps are accepted).

INFEASIBLE PRIMAL-DUAL IPM FOR LINEAR PROGRAMMING

Parameters

$\alpha_0 = 0.99$ a fraction-to-the-boundary stepsize factor;
 $\sigma \in (0, 1)$ barrier reduction parameter;
 $\varepsilon_p, \varepsilon_d, \varepsilon_o$ primal feasibility, dual feasibility and optimality tolerances:
 IPM stops when $\frac{\|\xi_p^k\|}{1+\|b\|} \leq \varepsilon_p$, $\frac{\|\xi_d^k\|}{1+\|c\|} \leq \varepsilon_d$ and $\frac{(x^k)^T s^k / n}{1+|c^T x^k|} \leq \varepsilon_o$.

Initialize IPM

iteration counter $k = 0$;
 primal-dual point $x^0 > 0, y^0 = 0, s^0 > 0$;
 barrier parameter $\mu^0 = (x^0)^T s^0 / n$;
 primal and dual infeasibilities $\xi_p^0 = b - Ax^0$ and $\xi_d^0 = c - A^T y^0 - s^0$.

Interior Point Method

while ($\frac{\|\xi_p^k\|}{1+\|b\|} > \varepsilon_p$ or $\frac{\|\xi_d^k\|}{1+\|c\|} > \varepsilon_d$ or $\frac{(x^k)^T s^k / n}{1+|c^T x^k|} > \varepsilon_o$) **do**

Update (reduce) the barrier $\mu^{k+1} = \sigma \mu^k$;

Solve the system (9): find the primal-dual Newton direction $(\Delta x, \Delta y, \Delta s)$.

Find $\alpha_p = \max\{\alpha : x^k + \alpha \Delta x \geq 0\}$ and $\alpha_d = \max\{\alpha : s^k + \alpha \Delta s \geq 0\}$;

Set $\alpha_p := \alpha_0 \alpha_p$ and $\alpha_d := \alpha_0 \alpha_d$;

Make a step

$x^{k+1} = x^k + \alpha_p \Delta x$;

$y^{k+1} = y^k + \alpha_d \Delta y$;

$s^{k+1} = s^k + \alpha_d \Delta s$.

Compute the infeasibilities: $\xi_p^{k+1} = b - Ax^{k+1}$ and $\xi_d^{k+1} = c - A^T y^{k+1} - s^{k+1}$;

Compute the complementarity gap: $\mu^{k+1} = (x^{k+1})^T s^{k+1} / n$;

Update the iteration counter: $k := k + 1$.

end-while

Infeasible Method. Newton's method applied to first-order optimality conditions of the barrier problem can easily deal with any primal or dual infeasibility. Making a step α in direction $\Delta = (\Delta x, \Delta y, \Delta s)$ obtained from (9) reduces the infeasibilities ξ_p and ξ_d to $(1 - \alpha)\xi_p$ and $(1 - \alpha)\xi_d$, respectively. In the usual behaviour of infeasible methods for LP, both primal and dual infeasibilities are reduced faster than the complementarity gap. It is hardly a surprise considering that the first two equations in (7) are linear and only the third one is nonlinear.

Central Path Neighbourhood. The $N_2(\theta)$ neighbourhood of the central path is very small and completely impractical. Any algorithm restricted to keep iterates in it would have to use conservative goals when reducing the complementarity gap (that is, σ very close to 1). Using $N_\infty(\gamma)$ or $N_S(\gamma)$ (see (13) and (14), respectively) provides much more room for iterates and allows to achieve significant reductions of the complementarity gap at each iteration.

Aggressive Optimization. Practical algorithms do not require σ to be very close to 1, but allow it to be any number from the interval $(0, 1)$ thus asking for a more significant reduction of the duality gap when solving Newton system (9). It is common to use target reductions $\sigma \in (0.01, 0.1)$ and indeed achieve average reductions $\mu_{k+1} / \mu_k \in (0.1, 0.5)$ per iteration.

The use of an infeasible algorithm, a wide neighbourhood and aggressive target reductions of the gap comes at the price of having to accept that steps in Newton direction need to be damped. The analysis of such algorithms is slightly more complicated and the worst-case complexity results are more pessimistic, usually guaranteeing convergence only in $\mathcal{O}(n \log(\frac{1}{\varepsilon}))$ iterations. However, these

methods display the best behaviour known to date and are used in almost all commercial as well as research implementations of IPMs. We have provided above a general algorithm of the infeasible primal-dual interior point method for linear programming.

We summarize this section by giving a brief perspective on the computational effort required by a standard IPM for LP and comparing it to that of the simplex method.

IPMs converge in a predictable and small number of iterations. Although the worst-case complexity suggests that $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ or $\mathcal{O}(n \log(\frac{1}{\epsilon}))$ iterations may be needed to reach optimality, in practice this number is almost constant and depends very little on the problem dimension. A reasonable estimate of *expected* number of IPM iterations seems to be $\mathcal{O}(\log n)$.

The simplex method is theoretically non-polynomial which means that there is no polynomial bound on the number of iterations it might require to solve an LP. Indeed, Klee and Minty [87] created a problem of dimension n , the solution of which requires 2^n iterations. However, in practice, the simplex method usually converges in a number of iterations which is $\mathcal{O}(m + n)$.

The methods differ hugely in the computational cost of a single iteration.

For IPM this is dominated by solving the Newton system (9) which is usually reduced to normal equations (11) and solved by forming a Cholesky factorization of matrix $A\Theta A^T$. Since large scale LPs are usually sparse the cost of the above operation strongly depends on the sparsity structure of A and the ability of the Cholesky implementation to exploit this sparsity on a given computer architecture.

The simplex method works with an LP constraint matrix of (1) partitioned into basic and nonbasic blocks $A = [B, N]$, where $B \in \mathcal{R}^{m \times m}$ is a nonsingular matrix and $N \in \mathcal{R}^{m \times (n-m)}$. All vectors of dimension n such as the primal variables x and dual slacks s (reduced costs in the language of simplex method) are partitioned accordingly. The simplex method exploits this partitioning to represent the vertices of the primal polytope as solutions of the following linear equation system:

$$\begin{bmatrix} B & N \\ 0 & I_{n-m} \end{bmatrix} \begin{bmatrix} x_B \\ x_N \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}. \quad (16)$$

Its solution $(x_B, x_N) = (B^{-1}b, 0)$ can easily be found assuming that there exists a (sparse) inverse representation of the basis matrix B . The steps of the method are limited to moves along the edges of the feasible polytope, that is, moves between the neighbour vertices. This corresponds to a swap of a single basic variable with a nonbasic one and the corresponding swap of a single column of basis matrix B with a column of N . There exist very efficient implementations of the basis inverse representation which allow for such a change and perform a fast update of the inverse representation [21,48,74,99,132]. A single iteration of the simplex method requires two backsolves with basis matrix B and one update of it after a column exchange. These operations involve an inverse representation of a (sparse) unsymmetric matrix B and usually may be executed in a small fraction of a second on modern computers.

In summary, IPM performs very few but possibly expensive iterations and the simplex method usually needs many iterations but those are very low-cost. Cholesky factorization takes advantage of parallelism while maintaining and updating the inverse representation of the basis in the simplex method is very difficult (practically impossible) to parallelise [73]. Unless some anomalies in the sparsity pattern of A occur, the larger the problem the more likely IPM is to win the race for efficiency. Additionally, for huge scale but specially structured LPs, there exist implementations of IPMs which can exploit massive parallelism available on modern high-performance computers [63,64,67].

Another important advantage of IPMs comes from their ability to deal with other difficult classes of optimization problems: quadratic, nonlinear, second order cone and semidefinite programming problems. We will discuss it in the next section.

4. Beyond linear programming

The use of a *nonlinear* logarithmic barrier function (3) to solve a *linear* optimization problem may seem counter-intuitive. Mathematicians commonly use local *linearization* as a tool to simplify a complicated nonlinear model or function, but the cases of deliberately “non-linearizing” the linear model are extremely rare. Indeed, in the author’s opinion, this may have been the reason why the LP community has not given the use of nonlinear barriers any serious consideration before Karmarkar’s breakthrough work was published [82]. Some of the advantages of logarithmic barrier functions have been known to the optimization community at least since the work of Frisch [53] which goes back to 1955. Their use in the context of nonlinear optimization [47] has been appreciated already in the 1960s. However, at that time the sizes of LPs were still too small to be able to demonstrate any computational advantage of what we today know as interior point methods. Another likely reason of not attempting to involve logarithmic barriers in LPs was the lack of efficient sparsity-exploiting Cholesky solvers. Those were only developed in the late 1960s and early 1970s [24,44].

The importance of the logarithmic barrier function in the development of IPMs has been clarified by Gill et al. [56], who demonstrated an equivalence between Karmarkar’s projective method and the projected Newton barrier method. Since then IPMs have occasionally been referred to as barrier methods. A natural next step was to use similar barriers to handle nonnegativity constraints in the context of quadratic programming [139] and to deal with more general inequalities in nonlinear programming [140]. Indeed, similarly to the logarithmic barrier function for nonnegativity constraints (3), for a more general nonlinear constraint $\{x \in \mathcal{R}^n, g(x) \leq 0\}$ we define $f : \mathcal{R}^n \rightarrow \mathcal{R}$

$$f(x) = \begin{cases} -\ln(-g(x)) & \text{if } g(x) < 0 \\ +\infty & \text{otherwise.} \end{cases}$$

For the quadratic cone $K_q = \{(x, t) : x \in \mathcal{R}^{n-1}, t \in \mathcal{R}, t^2 \geq \|x\|^2, t \geq 0\}$, we define $f : \mathcal{R}^n \rightarrow \mathcal{R}$

$$f(x, t) = \begin{cases} -\ln(t^2 - \|x\|^2) & \text{if } \|x\| < t \\ +\infty & \text{otherwise,} \end{cases}$$

and for the cone $SR_+^{n \times n}$ of positive definite matrices, we define $f : SR_+^{n \times n} \rightarrow \mathcal{R}$

$$f(X) = \begin{cases} -\ln \det X & \text{if } X \succ 0 \\ +\infty & \text{otherwise.} \end{cases}$$

The origin of this last logarithmic barrier function becomes clearer when one realises that a determinant of a positive definite matrix X satisfies $\det X = \prod_{j=1}^n \lambda_j$, where $\lambda_j, j = 1, \dots, n$ are the positive eigenvalues of X . Therefore

$$-\ln \det X = -\ln\left(\prod_{j=1}^n \lambda_j\right) = -\sum_{j=1}^n \ln \lambda_j$$

is just a logarithmic barrier function expected to keep all the eigenvalues λ_j strictly positive. The readers interested in more detail in interior point methods for nonlinear, second-order cone and semidefinite programming should consult excellent surveys of Forsgren et al. [49], Lobo et al. [95], and Vandenberghe and Boyd [136], respectively.

The exceptional features of the logarithmic barriers have been clarified by the theory of self-concordant functions developed by Nesterov and Nemirovski [116]. Let $C \in \mathcal{R}^n$ be an open nonempty convex set and let $f : C \rightarrow \mathcal{R}$ be a three times continuously differentiable convex function. A function f is called *self-concordant* if there exists a constant $p > 0$ such that

$$|\nabla^3 f(x)[h, h, h]| \leq 2p^{-1/2}(\nabla^2 f(x)[h, h])^{3/2},$$

$\forall x \in C, \forall h : x + h \in C$. (We then say that f is p -self-concordant). Note that a self-concordant function is always well approximated by the quadratic model because the error of such an approximation can be bounded by the $3/2$ power of $\nabla^2 f(x)[h, h]$.

The relevance of self-concordance becomes clearer when one looks in more detail at the IPM framework. In case of linear programming, IPMs rely on Newton method (9) applied to the first order optimality conditions (7) derived for the barrier problem (4). The Newton direction originates from solving the linear system which involves the second derivatives of the Lagrangian function for the barrier problem (4). Therefore, it involves the second derivative of the logarithmic barrier function, and the ability to contain the error in the Newton system, which is controlled by the next (third) derivative of the barrier function, is crucial in any convergence and worst-case complexity analysis.

All logarithmic barrier functions listed earlier in this section are self-concordant and IPM algorithms, which rely on these barriers, deliver $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ iteration worst-case complexity [116]. Obviously, there exist many other barrier functions and some of them might display certain advantages when applied in practice. However, unless they are also self-concordant, the proofs of convergence and the worst-case complexity analysis of the algorithms based on such barriers become more difficult.

The quest for finding the best possible barrier functions continues. In their highly influential work, Nesterov and Nemirovski demonstrated that each closed convex domain in $\mathcal{R}^n$ admits a *universal barrier* which employs an $\mathcal{O}(n)$ self-concordancy parameter and allows one to derive algorithms with the worst-case complexity $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ [116]. However, the iterations of an IPM based on the universal barrier are not efficiently computable. About a decade ago, Lee and Sidford [92] defined a self-concordant barrier based on Lewis weights [94], the so-called *Lewis weight barrier*. They used it to design an IPM algorithm with the worst-case iteration complexity $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$. This work has initiated a stream of papers which aim at lowering the theoretical computational complexity of IPMs for LP, see [34, 91–93] and the references therein. These efforts go in two directions and aim at clarifying the origin of the worst-case iteration complexity as well as reducing the computational cost of a single IPM iteration. Some other research directions involve specialised condition numbers [123, 131, 141] which shed light on computational challenges faced by IPMs.

The property of self-concordance requires convexity of the domain of the function, hence the strong results regarding the polynomial worst-case complexity are naturally restricted to convex optimization. This includes convex quadratic programming, convex conic programming and semidefinite programming. IPMs may also be applied in the absence of convexity, however, they come with weaker theoretical guarantees. Tackling general (nonconvex) nonlinear programming requires extra effort and at best a convergence to a local optimum may be established in such a case. IPMs were extended to nonlinear programming already in the 1990s [129, 140, 146, 147]. Since then solvers such as LOQO [139] and IPOPT [143] have been widely used and proved to be reliable and efficient in dealing with large scale problems.

The use of self-concordant barriers provides a unified theoretical framework for IPMs (for convex optimization). An equally important feature is a unified computational framework which is a consequence of the use of Newton's method which produces very similar linear equation systems for different classes of optimization problems. Compared with (10) in the LP case, when quadratic or inequality constrained nonlinear programming problems are solved with IPMs the augmented systems get more complicated. The matrices involved take the following forms:

$$\begin{bmatrix} -\Theta^{-1} & A^T \\ A & 0 \end{bmatrix}, \quad \begin{bmatrix} -Q - \Theta^{-1} & A^T \\ A & 0 \end{bmatrix}, \quad \begin{bmatrix} -Q(x, y) & A(x)^T \\ A(x) & \Theta \end{bmatrix}, \quad (17)$$

for linear, quadratic and nonlinear optimization problems, respectively. In (17), Q denotes the matrix which defines the quadratic objective function and $Q(x, y)$ and $A(x)$ denote the Hessian of the Lagrangian and the Jacobian of constraints in nonlinear optimization. The last two matrices change at each iteration of IPM for nonlinear programming. In case of linear programming the augmented system formulation (10) involves a diagonal scaling matrix Θ which permits reducing this system to a more compact normal equations

formulation (11). Mészáros [103] developed a clever heuristic to decide which of the two alternative approaches (10) or (11) is likely to produce a more sparse symmetric decomposition [104]. When quadratic or nonlinear problems are solved, the reduction of (17) to normal equations is usually not an option because it would destroy sparsity (an exception are separable quadratic problems when Q is diagonal).

The matrices in the augmented system formulation (17) are indefinite hence their symmetric factorizations LDL^T may require 2×2 pivots in D , see [24]. This is a significant complication when compared to the normal equation matrix $A\Theta A^T$ in LP case which is suitable for Cholesky factorization and can take advantage of any sparsity in A [44]. Vanderbei [138] observed that matrices in (17) are (or in case of LP or convex QP may be easily converted to) so-called *quasidefinite* matrices for which there is a guarantee that a decomposition LDL^T with diagonal D exists for any symmetric row and column permutation applied to the matrix. Hence there is no need to use 2×2 pivots and the reordering for sparsity performed at the beginning of optimization may be reused in all IPM iterations. The factorization of a quasidefinite matrix involves positive and negative entries in D and is therefore exposed to some additional difficulties with maintaining stability. Robust techniques based on regularization have been developed to address this issue [7,122,125].

It is worth mentioning that handling nonconvex optimization problems adds an extra difficulty when solving saddle point systems (17). For nonconvex problems, the $(1, 1)$ blocks ($Q + \Theta^{-1}$ and $Q(x, y)$ in the quadratic and nonlinear programming cases, respectively) may themselves be indefinite. In these cases a conversion of saddle point matrices to quasidefinite ones requires adding very large regularization terms to the $(1, 1)$ blocks and this may adversely affect the Newton IPM directions. An alternative approach is to use appropriate indefinite factorizations of saddle point matrices [44,126] and if it is possible to also exploit the identified negative curvature in the Hessian of the Lagrangian [49].

The computational linear algebra in IPMs for second order cone programming problems is very similar to that of LP and QP cases [10]. The handling of second-order conic constraints requires adding a rank-one corrector to the normal equation systems and if done carelessly it could destroy their sparsity. Efficient solvers for these problems, such as MOSEK [8,10], use the Sherman-Morrison-Woodbury formula to handle the rank-one update and prevent any degradation of sparsity in Cholesky factors. IPMs have also been extended to nonsymmetric convex conic optimization (see [115,130] and [120]).

The computational linear algebra operations required by an IPM for semidefinite programming problems [134,136] are significantly more complicated than those in LP or QP case. This is a consequence of dealing with a positive definite matrix $X \in \mathcal{SR}_+^{n \times n}$ which is a much more elaborate optimization variable than a vector $x \in \mathbb{R}^n$. The Lagrange multiplier associated with it is another matrix $S \in \mathcal{SR}_+^{n \times n}$, and the perturbed complementarity condition takes the form $XS = \mu I$. When the Newton system for the first order optimality conditions is derived the Jacobian matrix involves Kronecker products between $n \times n$ matrices and in general produces dense and very challenging matrices of dimension $n^2 \times n^2$. Moreover, the Newton system needs to be symmetrized [4,117,134] to guarantee that its solution delivers a symmetric matrix. Therefore, a realistic estimate of the computational effort of an IPM iteration for semidefinite programming is $\mathcal{O}(n^6)$. This is in contrast with the estimates of computational effort of IPM iteration for LP or QP for which a very pessimistic estimate is $\mathcal{O}(n^3)$ and it corresponds to solving a (rare in practice) dense LP or QP problem. Fortunately, many real-life large-scale semidefinite programming problems are structurally very sparse and their sparsity can be exploited in the solution of the Newton equations to avoid prohibitively expensive computations.

The estimates of the computational effort needed by a single IPM iteration are usually derived based on an assumption that *direct* methods of linear algebra are applied. Such estimates are often rather pessimistic because they apply to situations when dense factorizations may have to be computed. Any presence of sparsity in the data of the problem is considered to be a source of possible extra savings, but it remains hard to rigorously capture its impact and deliver improved worst-case complexity results.

However, direct methods of linear algebra may be prohibitively expensive at times, for example, when they suffer from excessive fill-in [44]. There also exist classes of problems in which matrices are not explicitly available or are too large to be stored or only the results of matrix-vector operations are available. All such situations call for an alternative approach such as the use of preconditioned *iterative* methods. This comes with new challenges, both theoretical and practical. We will address these issues in the next section.

5. Inexact Newton method and iterative solvers

Let us observe that a barrier problem (4) dealt with at a given IPM iteration does not have to be solved very exactly. IPM needs to make a quality Newton step towards its solution and noticeably reduce the complementarity gap, but this may well be achieved even if an *inexact* Newton method is used, that is, when the system (9) is solved ‘approximately’:

$$\begin{bmatrix} A & 0 & 0 \\ 0 & A^T & I_n \\ S & 0 & X \end{bmatrix} \cdot \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta s \end{bmatrix} = \begin{bmatrix} \xi_p \\ \xi_d \\ \xi_\mu \end{bmatrix} + \begin{bmatrix} r_p \\ r_d \\ r_\mu \end{bmatrix} \quad (18)$$

and admits controlled nonzero residuals r_p, r_d and r_μ in all three equations. This raises theoretical questions about how large the residuals may be without adversely affecting the convergence of IPM. Such a problem was addressed in several theoretical analyses [2,13,26,108]. In case of the feasible IPM algorithm, a somewhat surprising result [59] demonstrates that a significant level of residual r_μ is acceptable without any harm to the worst-case complexity of the method, which remains $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$. Namely, in the feasible IPM operating in $N_2(\theta)$ or $N_S(\gamma)$ neighbourhood, the residual r_μ may reach 30% or 5% of ξ_μ , respectively, and the degrees of polynomials in the corresponding worst-case complexity results remain unchanged in both cases. However, inexact infeasible IPMs have $\mathcal{O}(n^2 \log(\frac{1}{\epsilon}))$ iteration complexity [2].

Interior point methods play a significant role in the emerging area of *quantum computing* [11,85]. The use of quantum computers promises to deliver a spectacular acceleration of linear system solvers. This however is likely to come at the price of having to tolerate certain inexactness of the solutions they deliver. In this context, an ability to allow for the use of inexact Newton directions (within feasible [59] or infeasible [2] algorithm) seems crucial [109,110,145].

Allowing for nonzero residuals in (18) opens the door to the application of *iterative* methods [84] to compute Newton directions. Various techniques may be applied in this context including fixed-point iterations or Krylov-subspace methods. This provides a new feature of flexibility in the interior point algorithm and calls for new analyses of inexact methods which take into account the particular aspects of an iterative scheme to design appropriate stopping criteria and to analyse the impact of inexactness on the convergence of IPM. Indeed, iterative methods have the most to offer if they do not have to be run until reaching high accuracy. An example of such analysis has been reported recently [151] and demonstrated that standard termination criteria of iterative techniques might be too conservative. New stopping criteria to terminate the conjugate gradient and the minimum residual methods proposed in [151] have been shown to reduce the number of Krylov iterations by 60-80% and save 50-70% of computation time when compared to standard stopping criteria [84].

Iterative schemes are sensitive and may be slow on occasions: fixed-point iterations struggle when the spectral radius of the contraction mapping approaches 1, Krylov-subspace methods are challenged when condition number of the matrix grows. Newton systems (18) always become ill-conditioned when the IPM approaches an optimal solution. Indeed, for all indices j we expect $x_j s_j \approx \mu \rightarrow 0$. Hence for every index j , at least one of the two variables x_j or s_j has to approach zero causing very high rank deficiency of both matrices X and S . When the IPM approaches an optimal solution, primal variables and dual slacks converge to their optimal values and (for a linear program) converge to a strongly complementary pair [144, p. 27]:

$$\begin{aligned} x_j \rightarrow x_j^* > 0 \quad \text{and} \quad s_j \rightarrow s_j^* = 0, \quad \text{for } j \in \mathcal{B}, \\ x_j \rightarrow x_j^* = 0 \quad \text{and} \quad s_j \rightarrow s_j^* > 0, \quad \text{for } j \in \mathcal{N}. \end{aligned} \quad (19)$$

Although we borrowed the simplex method convention and used letters $\mathcal{B}$ and $\mathcal{N}$ here, the partitioning is usually very different from the simplex basic-nonbasic one (16). In particular the subset of indices in $\mathcal{B}$ is often larger than m . Even in case when it does contain exactly m indices, the corresponding subset of columns of matrix A usually does not define a nonsingular basis matrix in the sense of the simplex method. It is worth commenting at this point that the two methods (the simplex and IPMs) differ significantly w.r.t. degeneracy [72,148]. Simplex method applied to degenerate problem may cycle while the presence of degeneracy does not affect the iteration complexity of IPMs. However, for degenerate problems, when the $\mathcal{B}$ part of the optimal partition does not include a basis, the Newton system becomes nearly singular, which makes the linear algebra of IPM even more challenging.

The behaviour of x_j and s_j close to optimality affects the conditioning of the original block 3×3 formulation of Newton system (9). The ill-conditioning also propagates to the symmetric indefinite augmented system (10) and the symmetric positive definite normal equations (11) because the elements of diagonal matrix $\Theta = XS^{-1}$ display large variation of magnitude. The elements $\theta_j, j \in \mathcal{B}$ are very large $\theta_j = x_j/s_j = x_j^2/(x_j s_j) \approx \mathcal{O}(\mu^{-1})$, while the elements $\theta_j, j \in \mathcal{N}$ are very small $\theta_j = x_j/s_j = (x_j s_j)/s_j^2 \approx \mathcal{O}(\mu)$. The presence of the badly behaving matrix Θ makes both (10) and (11) numerically challenging. However, such systems have been studied extensively because very similar linear systems are at the heart of computational techniques for partial differential equations, engineering, statistics and optimization [16]. It is generally easier to deal with the ill-conditioned systems when direct methods of linear algebra are used [44]. Iterative methods are likely to fail miserably unless they are appropriately preconditioned.

An ideal preconditioner $P = EE^T$ of a given symmetric matrix H should (i) be easy to compute (significantly less expensive than any direct factorization of H), (ii) be easy to invert, and (iii) produce good spectral properties of the matrix $E^{-1}HE^{-T}$ (that is $P^{-1}H$). The third requirement may be translated into attempting to guarantee that $E^{-1}HE^{-T}$ has few distinct eigenvalues or has all eigenvalues in a small cluster: $\lambda_{\min} \leq \lambda \leq \lambda_{\max}$. In this context, a property displayed by the matrix Θ may be exploited to drop certain columns from systems (10) or (11) and make these systems sparser to deliver attractive preconditioners. Let us first illustrate it on the normal equations matrix. Using the outer product form of the matrix, we may exploit the fact that $\Theta_{\mathcal{N}} \rightarrow 0$, hence drop the contribution of the “nonbasic” variables and write

$$A\Theta A^T = \sum_{j=1}^n \theta_j A_j A_j^T = \sum_{j \in \mathcal{B}} \theta_j A_j A_j^T + \sum_{j \in \mathcal{N}} \theta_j A_j A_j^T \approx \sum_{j \in \mathcal{B}} \theta_j A_j A_j^T = E_1 E_1^T. \quad (20)$$

The above approximation uses a shorter loop to gather normal equations matrix and is likely to produce a quality approximation of $A\Theta A^T$, especially close to the optimal solution. Such a preconditioner is particularly attractive when $m \ll n$ and a large number of columns $j \in \mathcal{N}$ may be dropped. Indeed, it proved to be very well-suited to problems which arise from discrete optimal transport applications [33,150].

It is important to realise that the augmented system (10) offers an additional flexibility when compared with normal equations (11). Indeed, the latter has been produced by choosing a particular block pivot Θ^{-1} , which is equivalent to eliminating the variable Δx . It is well known in the community of sparse matrix factorizations that other pivot sequences might offer additional advantages and deliver sparser factors [103]. This property translates into an important feature for the design of preconditioners and it was first stated by Oliveira and Sorensen [119] who have shown that for every preconditioner of the normal equations, an equivalent preconditioner can be derived for the augmented system. However, the converse statement is not true. Therefore, a strong case exists that preconditioners should be developed for the augmented system (10).

There already exists a vast selection of preconditioners for symmetric indefinite systems arising in optimization [18,19,30,43,45,83,126]. Let us illustrate here one particular preconditioner developed by Oliveira [118,119] which exploits a guess of basic-nonbasic

partition derived from an intermediate IPM iteration. The preconditioner has the form EE^T with an easily invertible block triangular matrix E and therefore its symmetric application to the augmented system matrix from the left and from the right is an easy operation which gives the following result

$$\begin{aligned} E^{-1}HE^{-T} &= \left[\begin{array}{c|c} \Theta_B^{1/2} & \Theta_B^{-1/2}B^{-1} \\ \hline \Theta_N^{1/2} & \Theta_N^{1/2} \end{array} \right] \left[\begin{array}{c|c} -\Theta_B^{-1} & B^T \\ \hline -\Theta_N^{-1} & N^T \end{array} \right] \left[\begin{array}{c|c} \Theta_B^{1/2} & \Theta_B^{1/2} \\ \hline B^{-T}\Theta_B^{-1/2} & \Theta_N^{1/2} \end{array} \right] \\ &= \left[\begin{array}{c|c} I_m & W^T \\ \hline W & -I_{n-m} \end{array} \right], \end{aligned} \quad (21)$$

where $W^T = \Theta_B^{-1/2}B^{-1}N\Theta_N^{1/2}$. Following (19), sufficiently close to an optimal solution (when $\mu \rightarrow 0$) we have $\Theta_B^{-1} = \mathcal{O}(\mu) \rightarrow 0$ and $\Theta_N = \mathcal{O}(\mu) \rightarrow 0$ giving $W^T = \Theta_B^{-1/2}B^{-1}N\Theta_N^{1/2} \approx 0$ hence the preconditioned matrix is strongly dominated by its diagonal, and its eigenvalues are clustered around +1 and -1.

This particular preconditioner well illustrates an important aspect of a design of preconditioners in general. It delivers an advantageous structure of eigenvalues in matrix $E^{-1}HE^{-T}$ although the product EE^T does not seem to be a particularly good approximation of the matrix H itself (or $-H$ in this case). Indeed

$$P = EE^T = \left[\begin{array}{c|c} \Theta_B^{-1/2} & \Theta_B^{-1/2} \\ \hline B\Theta_B^{1/2} & -B\Theta_B^{1/2} \end{array} \right] \left[\begin{array}{c|c} \Theta_B^{1/2}B^T & \Theta_N^{1/2} \\ \hline -\Theta_B^{1/2}B^T & -\Theta_N^{1/2} \end{array} \right] = \left[\begin{array}{c|c} \Theta_B^{-1} & -B^T \\ \hline -B & 2B\Theta_B B^T \end{array} \right]$$

differs significantly from

$$-H = \left[\begin{array}{c|c} \Theta_B^{-1} & -B^T \\ \hline -B & -N \end{array} \right] \left[\begin{array}{c|c} \Theta_N^{-1} & -N^T \\ \hline -N & 0 \end{array} \right].$$

The preconditioners which need to identify a simplex-type nonsingular basis matrix are surprisingly difficult to construct in practice. Finding an approximate partitioning (19) does not imply that the subset of columns of A which correspond to B defines a full row rank matrix. Even if it does, a selection of exactly m columns from the set B such that the corresponding matrix delivers a stable basis is still a challenge. A number of heuristic techniques have been proposed to tackle the problem [3,22,119]. They perform well close to an optimal solution when the optimal partitioning is already revealed, but may struggle in the earlier IPM iterations. This may be remedied in two ways: either by using a different preconditioner in earlier IPM iterations [22] or by investing significantly more effort into identifying which columns should be used to construct a stable basis B .

The importance of choosing good basis matrix follows naturally from the analysis of (21). For the preconditioner to work well, the partitioning $A = [B|N]$ should guarantee that matrix $W^T = \Theta_B^{-1/2}B^{-1}N\Theta_N^{1/2} \approx 0$. A similar argument may be derived from the analysis of the normal equation matrix and expressing it in the following form

$$\begin{aligned} A\Theta A^T &= B\Theta_B B^T + N\Theta_N N^T \\ &= B\Theta_B^{1/2} \left(I + (\Theta_B^{-1/2}B^{-1}N\Theta_N^{1/2})(\Theta_N^{1/2}N^TB^{-T}\Theta_B^{-1/2}) \right) \Theta_B^{1/2}B^T \\ &= B\Theta_B^{1/2}(I + W^TW)\Theta_B^{1/2}B^T, \end{aligned}$$

which reveals that if W is small then $B\Theta_B B^T$ is a good approximation of $A\Theta A^T$.

To guarantee that the basis matrix is stable and delivers a good preconditioner Schork and Gondzio [127] suggested to use a maximum-volume concept when choosing the basis [14,70,71], that is, to select the basis which maximizes $\text{Vol}(B) = |\det(B)|$. Maximum volume of B implies that entries of $B^{-1}N$ cannot exceed 1. However, finding such a basis is an NP-hard problem. A relaxed version of the problem, that is, finding the so-called ρ -maximum volume basis which guarantees that any entry of matrix $\tilde{N} = B^{-1}N$ satisfies $|\tilde{N}_{ij}| \leq \rho$ for certain $\rho > 1$ can be computed very efficiently using simplex-type linear algebra techniques [74,127].

It is worth mentioning that choosing a stable basis for preconditioning purposes faces two potentially conflicting objectives. On one hand, we would like the choice to guarantee that the basis is constructed from columns with the largest possible values of θ_j thus helping to keep $W^T = \Theta_B^{-1/2}B^{-1}N\Theta_N^{1/2}$ small. On the other hand, we seek the ρ -maximum volume basis in A . A compromise (and an implementable heuristic) [127] is to find a ρ -maximum volume basis of the scaled matrix $\tilde{A} = A\Theta^{1/2}$. An efficient implementation of maximum volume based preconditioning has been developed in IPX solver [127] and is now available through an Open Source LP/QP software, HiGHS (<https://github.com/ERGO-Code/HiGHS>).

The difficulty originating from ill-conditioning of the matrices X and S in (9) and Θ in (10) and (11) makes the design of general-purpose preconditioners for IPMs challenging, although attempts to tackle the problem have already been made [17,32,55,66,119,127]. On the other hand, if the problem (1) reveals any particular property such as a special form of matrix A , a particular sparsity pattern of A , a presence of fast matrix-vector operator to deliver results of multiplications Ax and A^Ty , or any other *exploitable* feature, then such a property is very likely to contribute to a design of a clever bespoke preconditioner for an appropriately selected

iterative method. In contrast, exploiting any such information in a general direct method of sparse linear algebra may be impossible or tedious at best.

For example, there exist very powerful preconditioners for problems arising in network optimization [51,78,124] which exploit the spanning tree representation of simplex bases, and more recent ones [33,150] which rely on Laplacian representation of $A\Theta A^T$. There is a family of power-series preconditioners [27–29] which are able to tackle huge scale block-structured problems, such as multi-commodity network flow or multi-stage stochastic ones. We also expect more of developments of bespoke preconditioners for (possibly narrow) classes of problems which defy general-purpose IPM solvers. The list of examples of such developments already includes: compressed sensing problems [38,50], various sparse approximation and inverse problems [40,65], but is likely to grow fast in the near future. There has also been an attempt [31] to develop a randomised linear algebra technique and combine it with sketching to provide a preconditioner for LPs. However, such developments are limited to problems which have at least one dimension (m or n in (1)) really small.

A growing understanding of the role of iterative methods for linear algebra in IPMs [37] and the nature of ill-conditioning which appears especially in the later IPM iterations seem to be shifting the interest towards developing new IPMs with a low per-iteration cost. Having said that, due to the reliability and simplicity of accuracy control, the default commercial implementations of IPMs still seem to rely on direct sparse linear algebra solvers to compute Newton directions. This has not changed much from the 1990s [9,98] and is partly explained by an observation that regardless the ill-conditioning of matrices in the linear systems (10) and (11), their solutions remain uniformly bounded [42,131] and therefore sufficiently accurate Newton directions can still be produced.

6. Applying IPMs in decomposition and column generation

Although initially interior point methods were developed for linear programming, they were soon extended and excelled in delivering efficient, numerically stable and reliable solutions techniques also for other classes of optimization problems such as quadratic, nonlinear, second-order cone and semidefinite programming. IPMs are often default techniques chosen by commercial solvers, and more and more frequently also chosen by the users of commercial optimization packages. For an arbitrary large linear program, solvers such as Cplex, Gurobi or Mosek usually launch a race for efficiency on a multi-core computer. They run the primal simplex method and the dual simplex method each on a single core and the barrier method on available remaining cores. (IPM is able to take advantage of a multi-core parallelism.) The solution is delivered by the method which wins the race and, when the problem is large, IPM is usually the winner.

Integer and mixed-integer optimization still remain the areas where the impact of interior point methods has been smaller. The techniques for integer and combinatorial optimization involve branch and bound, column generation, cutting planes, and historically they have been entangled to the simplex method [54,114]. Branch and bound excels using an optimal basis to restart the solution of a closely related new LP relaxation and often reaches the new optimum in merely a few simplex iterations. Cuts which eliminate fractional solutions change the feasible polytope close to the previous optimal solution and again the new LP relaxation may often be solved efficiently if the simplex method starts from the previous optimal basis. This is particularly relevant if the number of newly added integrality cuts is small, but it may no longer be true when a large number of new constraints is generated and a significant change to the feasible polytope occurs. This is the situation which seems to favour IPMs.

The potential advantages of IPM-based solutions in this context were appreciated already in 1990s [105–107]. However, information about it does not seem to have propagated to the mainstream of the integer and combinatorial optimization communities. A possible explanation of that might originate from the perceived (but certainly unjustified) view that the theory, implementation and the use of IPMs are difficult. Another reason is a challenge related to the warmstarting of IPMs [58,62,149]. We will discuss potential advantages of IPMs when they are applied in two situations:

- to solve subproblems in Benders [15] and Dantzig-Wolfe [36] decompositions; and
- to solve restricted master problems in a cutting plane/column generation scheme.

For a start, let us once again draw the reader's attention to the notion of μ -centrality and its consequences for intermediate IPM solutions which are close to the central path. First of all, a primal and dual feasible solution which lies in the vicinity of the central path (i.e., it satisfies $XSe \approx \mu e$) is an approximate μ -centre (7) which delivers an excellent measure of the duality gap (8). Therefore it allows for an early termination of the solution process with a perfect control of a distance to optimality. This feature distinguishes IPM from the simplex method. The latter cannot deliver similar guarantees and can only be terminated when all reduced costs reach nonnegativity.

Subproblems in subsequent iterations of Benders decomposition [15] take the form of linear programs with changing right hand side vectors. In other words, there is a need of solving a sequence of LPs which share the same constraint matrix A but differ with vectors b . Each such subproblem is expected to provide a subgradient (an optimality cut) for the restricted master problem. Subproblems do not have to be solved to full accuracy; it is sufficient to find a primal and dual feasible solution with a guaranteed distance to optimality and use it to generate an ε -subgradient. This information is readily available in IPM because its iterates are primal and dual feasible and the distance to optimality is measured with (8). This cannot be achieved when running the simplex method, because this algorithm cannot be prematurely terminated with a guaranteed distance to optimality. (As long as at least one variable with a negative reduced cost exists, it is not possible to assess how far the current simplex iterate is from an optimal vertex.)

Subproblems in subsequent iterations of Dantzig-Wolfe decomposition [36] are linear programs with changing cost vectors. Similarly to the Benders decomposition case such subproblems do not have to be solved to optimality. Instead, an ε -optimal solution

may be found and an ε -subgradient may be delivered to the restricted master problem [68]. This allows to adjust the accuracy of subgradients to the needs of the upper level solver in the considered decomposition approach.

In summary, in both decomposition approaches subproblems do not have to be solved to high accuracy and the option to terminate IPM earlier is likely to deliver noticeable CPU time savings. An early termination feature available in IPM has an even greater importance when the restricted master problems are solved in a cutting plane/column generation scheme.

In the following two subsections we will address the solution of linear programming problems with matrices A displaying some special block-structure which may be exploited to decompose the problem and simplify its solution. Two most frequently met sparsity patterns are the dual and primal block-angular structures and they are dealt with the Benders and Dantzig-Wolfe decomposition algorithms, respectively. Then in the third subsection we will focus on the use of IPM in the context of cutting plane methods (or column generation). As an example one might consider the solution of restricted master problems arising in the two decomposition schemes mentioned earlier. Finally, in the fourth subsection, we will illustrate advantages of IPM-based column generation when solving very large discrete optimal transport problems.

6.1. Benders decomposition

Dual block-angular LPs arise in different applications, but the most common example originates from two-stage stochastic linear programming (see for example [79])

$$\begin{aligned} \min_{x \in X} \quad & c^T x + E_{\xi} \{q^T v(\xi)\} \\ \text{s.t.} \quad & T(\xi)x + Wv(\xi) = h(\xi), \\ & x \geq 0, \quad v(\xi) \geq 0, \quad \forall \xi \in \Xi, \end{aligned}$$

where E_{ξ} denotes the expected value with respect to the random variable $\xi \in \Xi$. If we assume that the random variable ξ has a finite discrete probability distribution $\{(\xi^i, \pi_i), i = 1, 2, \dots, p\}$ with $\sum_i \pi_i = 1$, the above problem may be written in a form of a stochastic linear program with fixed recourse:

$$\begin{aligned} \min_{x \in X} \quad & c^T x + \sum_{i=1}^p \pi_i q_i^T v_i \\ \text{s.t.} \quad & T(\xi^i)x + Wv_i = h(\xi^i), \quad i = 1, \dots, p, \\ & x \geq 0, \quad v_i \geq 0, \quad i = 1, \dots, p. \end{aligned}$$

After simplifying the notation $T(\xi^i) = T_i$ and $h(\xi^i) = h_i$, for $i = 1, \dots, p$, we write a *deterministic equivalent formulation*

$$\begin{aligned} \min_{x \in X} \quad & c^T x + \pi_1 q_1^T v_1 + \pi_2 q_2^T v_2 \quad \dots \quad + \pi_p q_p^T v_p \\ \text{s.t.} \quad & T_1 x + Wv_1 = h_1 \\ & T_2 x + Wv_2 = h_2 \\ & \vdots \quad \quad \quad \ddots \quad \quad \quad \vdots \\ & T_p x + Wv_p = h_p \\ & x \geq 0, \quad v_1 \geq 0, \quad v_2 \geq 0, \quad \dots \quad v_p \geq 0, \end{aligned}$$

which readily displays the dual block-angular structure.

The two-stage problem is difficult because the first-stage variables link all the blocks. If the first-stage variable could be *fixed* and (temporarily) set equal to x_0 , then having eliminated the constant term we would get a separable problem

$$\begin{aligned} \min \quad & \pi_1 q_1^T v_1 + \pi_2 q_2^T v_2 \quad \dots \quad + \pi_p q_p^T v_p \\ \text{s.t.} \quad & Wv_1 = h_1 - T_1 x_0 \\ & Wv_2 = h_2 - T_2 x_0 \\ & \quad \quad \quad \ddots \quad \quad \quad \vdots \\ & Wv_p = h_p - T_p x_0 \\ & v_1 \geq 0, \quad v_2 \geq 0, \quad \dots \quad v_p \geq 0, \end{aligned}$$

and solve it as p independent small LPs. The deterministic equivalent may also be written as

$$\min \{c^T x + \sum_{i=1}^p \pi_i Q_i(x) \mid x \geq 0\}, \tag{22}$$

where $Q_i(x)$, $i = 1, 2, \dots, p$, is the optimal objective function of the *recourse problem*

$$Q_i(x) = \min \{q_i^T v_i \mid Wv_i = h_i - T_i x, v_i \geq 0\}.$$

The function $\sum_{i=1}^p \pi_i Q_i(x)$ is piece-wise linear and convex, hence the problem (22), called the *master program*, may be solved using the cutting plane method. Depending on the value of the first-stage variable x , we distinguish two cases: (i) the subproblem i is *feasible* and has an optimal solution $\hat{v}_i$ and $Q_i(x) > -\infty$ and (ii) the subproblem i is *infeasible* hence we set $Q_i(x) = +\infty$. In both cases subproblem i will contribute a valid constraint (a cut) which will enrich the master problem. To simplify our discussion, let us assume that for any first-stage variable x , there exists a feasible solution of the recourse problem, that is, case (ii) does not happen. This may be guaranteed for example when the two-stage problem involves a complete recourse [79].

For a given first-stage decision x_0 , we will consider the following primal-dual pair of LPs:

$$\begin{array}{ll} \text{Primal} & \text{Dual} \\ \min & q_i^T v_i \\ \text{s.t.} & W v_i = h_i - T_i x_0, \\ & v_i \geq 0; \end{array} \quad \begin{array}{ll} \max & (h_i - T_i x_0)^T u_i \\ \text{s.t.} & W^T u_i \leq q_i, \\ & u_i \text{ free.} \end{array} \quad (23)$$

In case (i), when subproblem i is feasible, let $\hat{v}_i$ be the primal optimal solution and $\hat{u}_i$ be the dual optimal solution. From the strong duality we have

$$Q_i(x_0) = q_i^T \hat{v}_i = (h_i - T_i x_0)^T \hat{u}_i.$$

Then, using the dual, for any x we may write

$$Q_i(x) = \sup\{(h_i - T_i x)^T u_i \mid W^T u_i \leq q_i\}.$$

Thus for a given (feasible) $u_i = \hat{u}_i$ we construct the subgradient inequality for $Q_i(x)$:

$$Q_i(x) \geq (h_i - T_i x)^T \hat{u}_i, \quad \forall x.$$

Using $Q_i(x_0) = (h_i - T_i x_0)^T \hat{u}_i$ we can rewrite it:

$$Q_i(x) \geq Q_i(x_0) - \hat{u}_i^T T_i (x - x_0), \quad \forall x. \quad (24)$$

Hence $\delta_i = -T_i^T \hat{u}_i$ is a support of Q_i at x_0 , that is $-T_i^T \hat{u}_i$ belongs to the subdifferential of Q_i at x_0 . The inequality (24) is called an *optimality cut* for $Q_i(x)$.

Observe that if an IPM is used to solve subproblem i then there is no need to ask for an accurate solution. Instead, a primal-dual feasible and ϵ -suboptimal solution suffices to deliver an ϵ -optimality cut which may be used to replace the exact optimality cut (24). Once again let us assume that the first-stage decision x_0 is given and the primal-dual pair (23) needs to be solved. However, this time we assume that $(v_i^\epsilon, u_i^\epsilon)$ is the primal-dual feasible and ϵ -suboptimal solution of (23), that is, $W v_i^\epsilon = (h_i - T_i x_0)$, $W^T u_i^\epsilon \leq q_i$ and

$$(h_i - T_i x_0)^T \hat{u}_i \geq (h_i - T_i x_0)^T u_i^\epsilon \geq (h_i - T_i x_0)^T \hat{u}_i - \epsilon,$$

where $\hat{u}_i$ is an (unknown) exact optimal solution. Then using similar arguments as before, for an arbitrary x , we write:

$$\begin{aligned} Q_i(x) &\geq (h_i - T_i x)^T u_i^\epsilon \\ &= (h_i - T_i x_0)^T u_i^\epsilon - (u_i^\epsilon)^T T_i (x - x_0) \\ &\geq (h_i - T_i x_0)^T \hat{u}_i - \epsilon - (u_i^\epsilon)^T T_i (x - x_0) \\ &= Q_i(x_0) - \epsilon - (u_i^\epsilon)^T T_i (x - x_0), \end{aligned} \quad (25)$$

where $Q_i(x_0)$ is the exact (but unknown) optimal function value. Thus for an arbitrary point x_0 , we have the ϵ -subgradient inequality (25) which replaces the exact subgradient inequality (24), but can be found with less computational effort. Such ϵ -subgradient may be generated if an IPM is applied to solve subproblem i , but this would not be possible if the simplex method was applied in this context.

6.2. Dantzig-Wolfe decomposition

Consider a primal block-angular LP

$$\begin{array}{ll} \min & \sum_{i=1}^p c_i^T x_i \\ \text{s.t.} & \sum_{i=1}^p A_i x_i = a \\ & B_i x_i = b_i, \quad i = 1, 2, \dots, p, \\ & x_i \geq 0, \quad i = 1, 2, \dots, p, \end{array} \quad (26)$$

where $B_i \in \mathcal{R}^{m_i \times n_i}$, $A_i \in \mathcal{R}^{m_0 \times n_i}$, $i = 1, \dots, p$, $c_i, x_i \in \mathcal{R}^{n_i}$, $b_i \in \mathcal{R}^{m_i}$, $i = 1, \dots, p$ and $a \in \mathcal{R}^{m_0}$.

The problem would be easy without the coupling constraint $\sum_{i=1}^p A_i x_i = a$. Indeed, in such a case (26) becomes perfectly separable and may be replaced with p independent small LPs $\min_{x_i} \{c_i^T x_i \mid B_i x_i = b_i, x_i \geq 0\}$ for $i = 1, 2, \dots, p$.

We introduce Lagrange multiplier $y \in \mathcal{R}^{m_0}$ and associate with (26) the partial Lagrangian function

$$\begin{aligned} L(x, y) &= \sum_{i=1}^p c_i^T x_i + (\sum_{i=1}^p A_i x_i - a)^T y \\ &= -a^T y + \sum_{i=1}^p (c_i + A_i^T y)^T x_i. \end{aligned}$$

If the problem (26) has an optimal solution, we may, by duality, replace it with

$$\max_y L_D(y), \quad (27)$$

where $L_D(y) = \min_{x \in X} L(x, y)$ is the optimal value of

$$\begin{aligned} \min \quad & -a^T y + \sum_{i=1}^p (c_i + A_i^T y)^T x_i \\ \text{s. to} \quad & B_i x_i = b_i, \quad i = 1, \dots, p, \\ & x_i \geq 0, \quad i = 1, \dots, p. \end{aligned} \quad (28)$$

The Lagrangian dual function $L_D(y)$ is concave [20,35] but often nondifferentiable. In a particular case of dealing with (26) $L_D(y)$ is piece-wise linear and problem (27), called the *master program*, may be solved using a cutting plane method. This would require generating supergradients of $L_D(y)$. Problem (28) is called the *subproblem*. Observe that the feasible set of the subproblem is independent of y and that the Lagrangian dual function is *additive* and we may exploit its separability:

$$L_D(y) = -a^T y + \sum_{i=1}^p L_i(y), \quad (29)$$

where, for $i = 1, 2, \dots, p$

$$L_i(y) = \min_{x_i} \{(c_i + A_i^T y)^T x_i \mid B_i x_i = b_i, x_i \geq 0\}.$$

Therefore, subproblem (28) corresponds to the solution of p independent linear programs. The solution of each such problem delivers a supergradient for $L_i(y)$. A *disaggregation* of (29) allows for more accurate approximation of the function $L_D(y)$ and usually leads to a faster solution of the problem. Observe that if the original problem (26) is feasible then so is each of the above p problems. However, depending on the chosen value of Lagrange multiplier y , in general we would distinguish two cases: (i) the subproblem i has an optimal solution $\hat{x}_i$ and a *finite* objective function value $L_i(y) > -\infty$ and (ii) the subproblem i has an unbounded ray $\hat{r}_i$ and an *infinite* objective function value $L_i(y) = -\infty$. To simplify our discussion, let us assume that the feasible set of each subproblem is bounded and therefore case (ii) cannot happen.

We will construct polyhedral (piece-wise linear) approximations of each function $L_i(y)$. Every optimal solution of subproblem i for a particular value of y will provide a supergradient to $L_i(y)$. Indeed, let $y = \bar{y}$ and assume that every subproblem i has an optimal solution $\hat{x}_i$, for $i = 1, 2, \dots, p$. Since the feasible sets of all subproblems are independent of y , point $\hat{x}_i$ is a feasible solution for subproblem i for any value of y . Thus, for an arbitrary y , we have the supergradient inequality

$$\begin{aligned} L_i(y) &\leq (c_i + A_i^T y)^T \hat{x}_i \\ &= (c_i + A_i^T \bar{y})^T \hat{x}_i + (A_i \hat{x}_i)^T (y - \bar{y}) \\ &= L_i(\bar{y}) + (A_i \hat{x}_i)^T (y - \bar{y}), \end{aligned} \quad (30)$$

meaning that every function L_i has its own support $\delta_i = A_i \hat{x}_i$ at the point $\bar{y}$.

It may be unnecessary (and wasteful) to insist on solving each subproblem to perfect optimality, especially in the early stages of the decomposition algorithm. If an IPM is used to solve subproblem i , then finding an approximate optimal solution with a guaranteed small duality gap is sufficient to deliver an ϵ -supergradient inequality. Indeed, let $y = \bar{y}$ and assume that the IPM generates a primal and dual feasible pair with a duality gap less than ϵ , that is, the approximate solution x_i^ϵ is primal feasible and satisfies

$$(c_i + A_i^T \bar{y})^T \hat{x}_i \leq (c_i + A_i^T \bar{y})^T x_i^\epsilon \leq (c_i + A_i^T \bar{y})^T \hat{x}_i + \epsilon,$$

where $\hat{x}_i$ is an (unknown) exact optimal solution. Then, using similar arguments as before, for an arbitrary y , we write:

$$\begin{aligned} L_i(y) &\leq (c_i + A_i^T y)^T x_i^\epsilon \\ &= (c_i + A_i^T \bar{y})^T x_i^\epsilon + (A_i x_i^\epsilon)^T (y - \bar{y}) \\ &\leq (c_i + A_i^T \bar{y})^T \hat{x}_i + \epsilon + (A_i x_i^\epsilon)^T (y - \bar{y}) \\ &= L_i(\bar{y}) + \epsilon + (A_i x_i^\epsilon)^T (y - \bar{y}), \end{aligned} \quad (31)$$

where $L_i(\bar{y})$ is the exact (but unknown) optimal function value. Thus for an arbitrary point $\bar{y}$, we have the ϵ -supergradient inequality (31) which replaces the exact supergradient inequality (30), but can be found with less computational effort. Such ϵ -supergradient may be generated if an IPM is applied to solve subproblem i , but this would not be possible if the simplex method was applied in this context.

6.3. Restricted master problems in cutting plane/column generation schemes

The decomposition techniques presented in the previous two subsections are heavily used in the context of integer and combinatorial optimization. Dantzig-Wolfe decomposition is at the heart of column generation and the Branch-Cut-and-Price scheme [137]. Benders decomposition is commonly used in mixed integer optimization including cases which originate from multistage stochastic programming [23]. In both decomposition schemes there is a need to solve *master programs* (22) and (27). Both are well-defined convex piece-wise linear problems. (To be precise: (22) is a minimization of a convex function and (27) is a maximization of a concave function.) The functions are not known explicitly; instead, they are constructed by appropriate *oracles* in a form of subproblems which generate sub- or super-gradient information in a form of *optimality cuts*. As was demonstrated in the two previous subsections, if subproblems are solved using IPMs, the early termination with ϵ -accuracy may be exploited to deliver ϵ sub- or super-gradient information, respectively.

Convex optimization problems with piece-wise linear objective functions and linear constraints may be easily transformed to linear programs [35]. In subsequent iterations of decomposition schemes, problems (22) and (27) would grow in size because all new appended optimality cuts (whether exact or ϵ -accurate) contribute new linear constraints which improve the piece-wise linear model of the objective function. In both cases the objective functions have additive structure (see (22) and (29), respectively) and it is advantageous to disaggregate them and generate multiple optimality cuts, one for each of the p subproblems. However, this means that many new inequality constraints are added at each major iteration and in consequence the master problems quickly grow in size.

Since optimality cuts have the form of inequalities, we will assume that master problems (22) and (27) (see also (29)) are represented as *dual* LPs in (1). Indeed, the dual feasibility constraint in our standard formulation involves the dual slack variable s and may be naturally written in an inequality form $A^T y \leq c$. In other words, we will assume that optimality cuts (24) and (30) (or their ϵ -accurate versions (25) and (31), respectively) are simply contributing new dual constraints of the following form

$$A_j^T y \leq c_j, \quad j \in N = \{1, 2, \dots, n\}.$$

The expected large number of cuts to be considered means that the primal problem is likely to have a warehouse shape, that is, $A \in \mathbb{R}^{m \times n}$ with $m \ll n$. Linear programs which display such a feature are eligible to *column generation* scheme in which a “long” master problem (MP) is replaced by a sequence of “shorter” restricted master problems (RMPs) each of which involves a carefully selected subset of columns $\bar{N} \subset N$:

Master LP	Restricted Master LP	
$\min \quad \sum_{j \in N} c_j x_j$	$\min \quad \sum_{j \in \bar{N}} c_j x_j$	(32)
$\text{s.t.} \quad \sum_{j \in N} A_j x_j = b$	$\text{s.t.} \quad \sum_{j \in \bar{N}} A_j x_j = b$	
$x_j \geq 0, \quad \forall j \in N;$	$x_j \geq 0, \quad \forall j \in \bar{N}.$	

In standard column generation algorithms [12,96,137], a particular instance of restricted master problem is solved to optimality. Next, the corresponding dual optimal solution $\hat{y}_{rmp}$ is used to price all (or a subset of) the missing columns by computing

$$s_j = c_j - A_j^T \hat{y}_{rmp}, \quad j \in N \setminus \bar{N},$$

and the *value of the oracle* is defined as $z_O(\hat{y}_{rmp}) := \min\{s_j | j \in N \setminus \bar{N}\}$. In various applications of column generation in the combinatorial optimization context, the pricing problem might involve a significantly more sophisticated operation than just a single scalar product. It might actually require solving a nontrivial subproblem which depends on a current query point $\hat{y}_{rmp}$ and delivers new column information c_j and A_j . Columns for which $s_j < 0$ are candidates to be appended to the subsequent restricted master problem. Usually only some of them are added, for example the ones which deliver the most negative values of s_j . In the Benders and Dantzig-Wolfe decomposition schemes, “pricing” information is delivered by solving the corresponding subproblems. In these applications it is natural to append one cut (generate one column) per each of disaggregated subproblems, that is, to find sub- or super-gradients for every term $Q_i(x)$ and $L_i(y)$ in (22) and (29), respectively.

Significant advantages arise when an interior point method is used to solve the RMP. It seems unnecessary (and possibly wasteful) to solve RMPs to optimality knowing that potentially many new columns will have to be added in the coming iterations, and RMP will be subjected to significant changes. Indeed, IPM may be terminated earlier and deliver an approximate solution of RMP with a guaranteed duality gap. Assume $(x_{rmp}^\epsilon, y_{rmp}^\epsilon)$ is such a primal and dual feasible and ϵ -optimal solution, that is,

$$\sum_{j \in \bar{N}} c_j (x_{rmp}^\epsilon)_j - b^T y_{rmp}^\epsilon \leq \epsilon. \quad (33)$$

Then a feasible solution to the master problem is easily constructed by setting

$$x_j = \begin{cases} (x_{rmp}^e)_j & \text{if } j \in \bar{N}, \\ 0 & \text{if } j \in N \setminus \bar{N}. \end{cases}$$

The value of the primal objective at this point $\bar{z}^e = \sum_{j \in \bar{N}} c_j (x_{rmp}^e)_j$ is an *upper* bound (UB) on the optimal solution of the master problem.

Assume that a constant κ is known such that $\kappa \geq \sum_{j \in N} \hat{x}_j$, where $\hat{x}$ is an optimal solution of the MP. Since we cannot reduce the optimal value of RMP by more than κ times $z_O(\hat{y}_{rmp})$, the expression $\bar{z}^e + \kappa z_O(\hat{y}_{rmp})$ delivers a *lower* bound (LB) on the optimal solution of the master problem. Hence the optimal function value of the master $\hat{z}_{MP}$ satisfies

$$\hat{z}_{RMP} + \kappa z_O(\hat{y}_{rmp}) \leq \hat{z}_{MP} \leq \hat{z}_{RMP}.$$

When an ϵ -optimal solution of RMP $\bar{z}^e$ is known, the above inequality is replaced with

$$\bar{z}^e - \epsilon + \kappa z_O(y_{rmp}^e) \leq \hat{z}_{MP} \leq \bar{z}^e.$$

Below we summarise the above discussion and give the algorithm of the Primal-Dual Column Generation Method (PDCGM). For its detailed description the reader is referred to [60,61]).

This algorithm differs from the standard column generation in the way how RMPs are solved. Instead of solving them to optimality, we only ask for an *approximate* optimal solution. The requested relative accuracy of RMP, ϵ_r , depends on the relative duality gap reached so far for the MP (see line 6). It is advisable to use $D \in (2, 10)$ meaning that the solution of RMP needs to deliver more accuracy to make sure that bounds of the MP are systematically tightened and eventually convergence is reached. It is important to avoid unnecessary over-solving of RMPs, especially in the early stages of the column generation process when only a rough information is available and pressing for optimality is a waste of computational effort. Another crucial feature of PDCGM is the use of *well-centred* solutions of the RMPs. Such solutions can be effortlessly delivered by an interior point method [58,60] and have beneficial stabilising effect for the evolution of dual solutions $\{y_{rmp}^e\}_k$ produced by a sequence of RMPs, thus preventing column generation from a danger of *zig-zagging* (an effect often experienced in the standard column generation which relies on the use of simplex method to solve RMPs).

PRIMAL-DUAL COLUMN GENERATION METHOD

Parameters

ϵ_{max} = 0.99 maximum relative optimality tolerance for RMP;
 ϵ_{opt} relative optimality tolerance for MP;
PDCGM stops when $gap = (UB - LB)/(1 + |UB|) \leq \epsilon_{opt}$;
 $D > 1$ degree of optimality;

Initialize PDCGM

iteration counter $k = 0$;
 $UP = \infty$;
 $LB = -\infty$;
 $gap = \infty$;
 $\epsilon_r = 0.5$ (dynamic) relative RMP accuracy;

PDCGM

while ($gap > \epsilon_{opt}$) **do**

1. find a well-centred suboptimal solution (x_{rmp}^e, y_{rmp}^e) of the RMP;
2. $UB = \min\{UB, \bar{z}^e\}$;
3. call the oracle with the query point y_{rmp}^e ;
4. $LB = \max\{LB, \bar{z}^e - \epsilon + \kappa z_O(y_{rmp}^e)\}$;
5. $gap = (UB - LB)/(1 + |UB|)$;
6. $\epsilon_r = \min\{\epsilon_{max}, gap/D\}$;
7. if $(z_O(y_{rmp}^e) < 0)$ then add new columns to the RMP;

end-while

To explain better the notion of well-centred solutions produced by an interior point method let us come back to some of the basic properties of IPMs. Let $(x(\mu), y(\mu), s(\mu))$, $x(\mu) > 0$, $s(\mu) > 0$ be a μ -centre associated with a particular value of barrier parameter μ , that is, an exact solution of (7). To simplify the notation we will drop out the argument μ and write $(\hat{x}, \hat{y}, \hat{s})$ to denote an *exact* μ -centre. Observe that both $\hat{x}$ and $\hat{s}$ are strictly positive and therefore the IPM scaling matrix $\hat{\Theta} = \hat{X}\hat{S}^{-1}$ is well defined. With such an exact μ -centre $(\hat{x}, \hat{y}, \hat{s})$ and the corresponding primal-dual scaling $\hat{\Theta} = \hat{X}\hat{S}^{-1}$ we associate the following ellipsoid in the dual space:

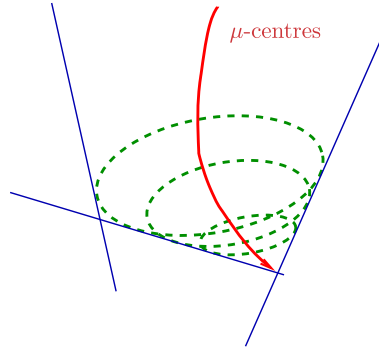


Fig. 1. Dikin's ellipsoids.

$$\hat{E} = \{y \in \mathcal{R}^m : \|\hat{\Theta}^{1/2} A^T(y - \hat{y})\| \leq \mu^{1/2}\},$$

which is displayed in Fig. 1.

Proposition 1. *If a point y belongs to the dual ellipsoid defined above then it is dual feasible. In other words*

$$\hat{E} \subset \mathcal{D} = \{y \in \mathcal{R}^m : A^T y \leq c\}.$$

Proof. The point $(\hat{x}, \hat{y}, \hat{s})$ is clearly primal-dual strictly feasible hence $A^T \hat{y} + \hat{s} = c$ and $\hat{s} > 0$. Consider another point $(x, y, s) \in \hat{E}$ and choose s such that $A^T y + s = c$. We do not know if this point is dual feasible, that is, if $s \geq 0$. We will show it is. Since $A^T \hat{y} + \hat{s} = c$ and $A^T y + s = c$ we deduce that $A^T(y - \hat{y}) + s - \hat{s} = 0$. Hence

$$\|\hat{\Theta}^{1/2} A^T(y - \hat{y})\|^2 = (y - \hat{y})^T A \hat{X} \hat{S}^{-1} A^T(y - \hat{y}) = (s - \hat{s})^T \hat{X} \hat{S}^{-1}(s - \hat{s}).$$

Observe that $\hat{X} = \mu \hat{S}^{-1}$ because $(\hat{x}, \hat{y}, \hat{s})$ is a μ -centre. Hence $\hat{X} \hat{S}^{-1} = \mu \hat{S}^{-2}$ so, the ellipsoid inequality may be rearranged as

$$\mu(s - \hat{s})^T \hat{S}^{-2}(s - \hat{s}) \leq \mu,$$

which is equivalent to $\|\hat{S}^{-1}(s - \hat{s})\|_2 \leq 1$. This implies:

$$\max_j \left\{ \frac{(s_j - \hat{s}_j)}{\hat{s}_j} \right\} = \|\hat{S}^{-1}(s - \hat{s})\|_\infty \leq \|\hat{S}^{-1}(s - \hat{s})\|_2 \leq 1$$

and yields $0 \leq s_j \leq 2\hat{s}_j, \forall j$, which guarantees that (y, s) is dual feasible. $\square$

In practice, IPMs do not find *exact* μ -centres. Instead, they keep the iterates in the vicinity of the central path and find *approximate* μ -centres. Depending on the type of the method used, we may consider different neighbourhoods of the central path, see for example (12) or (14).

For the $N_2(\theta)$ neighbourhood (12), let $(\bar{x}; \bar{y}, \bar{s})$ be an approximate μ -centre

$$\|(\bar{X} \bar{S} e / \mu) - e\| \leq \theta \leq 1,$$

and let

$$\bar{E} = \{y \in \mathcal{R}^m : \|\bar{\Theta}^{1/2} A^T(y - \bar{y})\| \leq \mu^{1/2}(1 - \theta)^{1/2}\}$$

be the corresponding ellipsoid. It is easy to observe that $\bar{E}$ approaches $\hat{E}$ when θ approaches zero. Using similar arguments, and assuming that $(\bar{x}; \bar{y}, \bar{s})$ belongs to the symmetric neighbourhood $N_S(\gamma)$ (see (14)) we define another ellipsoid

$$\tilde{E} = \{y \in \mathcal{R}^m : \|\tilde{\Theta}^{1/2} A^T(y - \bar{y})\| \leq \mu^{1/2} \gamma^{1/2}\}$$

and observe that $\tilde{E}$ approaches $\hat{E}$ when γ approaches 1.

We conclude this brief discussion with the following statement.

Proposition 2.

$$\hat{E}, \bar{E}, \tilde{E} \subset \mathcal{D} = \{y \in \mathcal{R}^m : A^T y \leq c\}.$$

We skip the proof of it because it is similar to that for the exact ellipsoid. Let us observe that all these ellipsoids define well shaped regions inside the dual feasible set. Any points in such ellipsoids are guaranteed to be kept away from the boundary of the dual

feasible region. Consequently, they contain rich information about the shape of the polytope close to the optimal solution. No such information is available when the simplex method is applied to solve the problem.

Coming back to the primal-dual column generation method, its ability to find *well-centred* solutions of RMP (x_{rmp}^e, y_{rmp}^e) with perfect control of the distance to optimality stabilises the behaviour of the method. Computational results reported in [60,61] provide clear evidence of such benefits.

Column generation is a major building block of the Branch-Price-and-Cut scheme widely applied in the context of solving mixed-integer programs [12]. This scheme has been shown to benefit from the use of the interior point method and the PDGM [112]. However, existing habits and decades of experience of relying on the simplex method embedded in column generation scheme result in certain reluctance of the mixed-integer optimization community to embrace the new IPM-based methodology.

We will conclude our discussion of advantages delivered by IPMs in decomposition/cutting planes/column generation with an example of using column generation in the context of continuous optimization arising in discrete optimal transport.

6.4. IPMs and column generation in discrete optimal transport

A discrete optimal transport problem [121] is closely related to a well-studied problem of finding a minimum cost flow through the network [1]. For large graphs with high degrees of connectivity such problems lead to very large linear programs with constraint matrices $A \in \mathbb{R}^{m \times n}$ which possess the property $m \ll n$. This makes them well-suited to an application of column generation so we will illustrate advantages of using IPM in this context.

Consider the Kantorovich [80] formulation of the discrete Optimal Transport (OT) problem: given a starting mass configuration $f \in \mathbb{R}_+^{n_1}$ and a destination configuration $g \in \mathbb{R}_+^{n_2}$, such that $\sum_{k=1}^{n_1} f_k = \sum_{l=1}^{n_2} g_l$, move the mass from f to g , that is, find a coupling (transport) matrix $\mathcal{T}$ inside the set of feasible transportation plans

$$U(f, g) = \left\{ \mathcal{T} \in \mathbb{R}_+^{n_1 \times n_2}, \mathcal{T} e_{n_2} = f, \mathcal{T}^T e_{n_1} = g \right\},$$

which is optimal with respect to a certain cost matrix $C \in \mathbb{R}_+^{n_1 \times n_2}$; i.e. find the solution of the following optimization problem

$$\min_{\mathcal{T} \in U(f, g)} \sum_{k,l} C_{kl} \mathcal{T}_{kl}.$$

This OT problem may be rewritten as a standard LP:

$$\begin{aligned} \min \quad & c^T t \\ \text{s.t.} \quad & \begin{bmatrix} e_{n_2}^T \otimes I_{n_1} \\ I_{n_2} \otimes e_{n_1}^T \end{bmatrix} t = \begin{bmatrix} f \\ g \end{bmatrix}, \\ & t \geq 0, \end{aligned} \tag{34}$$

where $\otimes$ denotes the Kronecker product, $c \in \mathbb{R}^{n_1 n_2}$ and $t \in \mathbb{R}^{n_1 n_2}$ are the vectorized versions of C and $\mathcal{T}$, respectively, $c = \text{vec}(C)$ and $t = \text{vec}(\mathcal{T})$. Assuming that there are no restrictions how mass may be moved, that is, it may be shipped from any location $k \in \{1, 2, \dots, n_1\}$ to any location $l \in \{1, 2, \dots, n_2\}$, this LP has $n_1 + n_2$ constraints and $n_1 \times n_2$ variables. The constraint matrix of (34) clearly displays a “warehouse” shape and is well-suited to a direct application of column generation [33,150].

The discrete OT problem has a natural interpretation as network optimization problem. Indeed, we consider a bipartite graph with vertices in $\mathcal{V}_1$ and $\mathcal{V}_2$ and edges (v_k, v_l) such that $v_k \in \mathcal{V}_1$ and $v_l \in \mathcal{V}_2$, and we are looking for a minimum cost flow from supply vertices $\mathcal{V}_1$ to demand ones $\mathcal{V}_2$. In other words, we wish to move the mass spread across vertices $\mathcal{V}_1$ to locations spread across vertices $\mathcal{V}_2$. In this interpretation, transport decisions $\mathcal{T}_{kl}$ are flows from vertex $v_k \in \mathcal{V}_1$ to vertex $v_l \in \mathcal{V}_2$ and C_{kl} is a cost coefficient associated with this flow.

The constraint matrix $A \in \mathbb{R}^{(n_1+n_2) \times (n_1 n_2)}$ in (34) is a vertex-edge incidence matrix, hence every column A_j in it has merely two nonzero entries. Although matrix A is very sparse, the adjacency structure AA^T which appears in (11) is actually quite dense. This is a consequence of high connectivity of a graph considered in a general discrete OT problem where mass may be moved from any vertex in $\mathcal{V}_1$ to any vertex in $\mathcal{V}_2$. The same does not have to hold when the graph is *sparse*. In this case A does not have to be such an extremely “long” matrix and adjacency structure AA^T might also enjoy some sparsity.

The main computational effort of IPM for a discrete optimal transport problem (34) consists of building $A\Theta A^T$ where Θ is a diagonal matrix and then solving equations with this matrix (see (11)). There are straightforward computational time savings resulting from replacing a complete problem (34) (which is equivalent of a master problem in this context) with a “shorter” version of it which involves only a subset of edges in the graph (and corresponds to a restricted master problem). Using the notation of column generation (32) this amounts to replacing the long summation $\sum_{j=1}^N \theta_j A_j A_j^T$ with a shorter one $\sum_{j=1}^{\tilde{N}} \theta_j A_j A_j^T$.

Despite obvious benefits of using a shorter summation loop, there are expected additional savings resulting from improved sparsity of the normal equations corresponding to RMP when compared to those of a full MP. It is worth observing that the interpretation of A as a vertex-edge incidence matrix of a graph implies that matrix $A\Theta A^T$ has a Laplacian structure [51,52,142] hence it is diagonally dominant. When it is additionally regularized it becomes strongly diagonally dominant and can be well approximated (and easily preconditioned) [33].

Column generation scheme has been combined with iterative linear algebra techniques to solve very large discrete optimal transport problems in [33,150]. Computational results provide evidence that IPM-based approach can efficiently solve such problems and

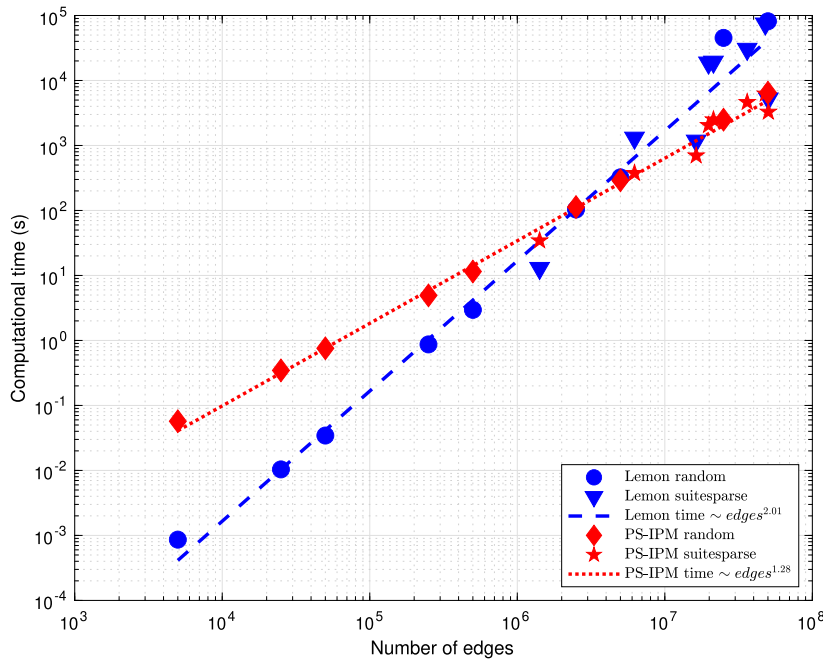


Fig. 2. Comparison of IPM vs LEMON on Random and SuiteSparse graphs.

outperforms the specialised OT solvers [121], specialised versions of network simplex method [77] and sophisticated network flow solvers of LEMON [90]. We refer the reader to [33,150] for detailed comparisons of various solvers for both dense and sparse graphs.

To conclude this section let us illustrate the performance of IPM combined with column generation on a set of large and sparse graphs. Examples include randomly generated graphs from [46] and real-life ones from the SuiteSparse matrix collection [39]. The randomly generated graphs have between 10^3 and 10^7 vertices and the SuiteSparse graphs have between 3×10^5 and 5×10^7 vertices. All graphs are sparse with average vertex degrees in the range [1, 10].

In Fig. 2 we report solution times of our specialised interior point solver PS-IPM and LEMON [90] for these graphs. The results reveal that when sizes of graphs exceed about 1 million of edges, PS-IPM is faster than LEMON. The regression lines are drawn for the two compared solvers to demonstrate a trend of solution times when the sizes of graphs increase. Since the sizes of the considered graphs as well as the solution times differ by several orders of magnitude, we use logarithmic scales for both. A straight line in such case represents an exponential dependence of the CPU time on the problem size and the slope of the line corresponds to the exponent. Hence we estimate that $t \approx Cn^\omega$, where t is the solution time, n is the problem size (number of edges in the graph) and C is a constant. The key interest here is of course the exponent ω which reflects the “observed” computational complexity of the two compared solvers. The estimated exponents ω are 2.01 for LEMON, and 1.28 for IPM, respectively.

To put these numbers in the right perspective, it is worth comparing them with the exponents which appear in the worst-case complexity analysis results and the estimated expected (or usually observed) exponents. The best known to date worst-case iteration complexity of IPMs is $\mathcal{O}(\sqrt{n} \log(\frac{1}{\epsilon}))$ (delivered by the short-step method). Each iteration requires a factorization of the matrix $A\Theta A^T$ which costs $\mathcal{O}(n^3)$ when dense linear algebra is used. Hence the worst-case computational complexity of IPM for LP is $\mathcal{O}(n^{3.5} \log(\frac{1}{\epsilon}))$. It is common to see a significantly lower number of iterations needed by practical implementations of IPMs, reasonably estimated with $\mathcal{O}(\log n)$ with the average cost of a single iteration varying between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ depending on the level of sparsity in the constraint matrix A and a (difficult to predict) level of sparsity in the Cholesky factor of $A\Theta A^T$.

The estimate of an exponent of n which is observed in the computational results obtained with IPM and reported in Fig. 2 seems very low indeed and may be hard to match by any method for discrete optimal transport or network flow problems.

7. Conclusions

We have addressed in this paper several important aspects of the theory, design and implementation of interior point methods. Their impact on the field of optimization has been very significant indeed. IPMs have delivered practical algorithms for solving several classes of optimization problems including linear, quadratic, nonlinear, second-order cone and semidefinite programming ones.

Although in 2024 they have officially reached the middle age of 40 years (since Karmarkar published his seminal paper [82]) they certainly have built on various developments made before 1984 including the works demonstrating the benefits of logarithmic barrier functions [53], the important role of centres in optimization [25,76], the use of affine-scaling directions and the resulting implicit projective scaling of the feasible polytope [42]. In this sense, similarly to many other methods of Operational Research, they have capitalised on more than 50 years of OR experience.

Declaration of competing interest

I hereby declare that my paper does not involve any conflict of interests.

Acknowledgement

The author is grateful to Dr Stefano Cipolla and Dr Filippo Zanetti for reading earlier version of this paper and useful suggestions which led to an improvement of the presentation.

References

- [1] R.K. Ahuja, T.L. Magnanti, J.B. Orlin, *Network Flows*, Prentice-Hall, New York, 1993.
- [2] G. Al-Jeiroudi, J. Gondzio, Convergence analysis of inexact infeasible interior point method for linear optimization, *J. Optim. Theory Appl.* 141 (2009) 231–247.
- [3] G. Al-Jeiroudi, J. Gondzio, J. Hall, Preconditioning indefinite systems in interior point methods for large scale linear optimization, *Optim. Methods Softw.* 23 (2008) 345–363.
- [4] F. Alizadeh, J.-P. Haeberly, M. Overton, Primal-dual interior-point methods for semidefinite programming: convergence rates, stability and numerical results, *SIAM J. Optim.* 8 (1998) 746–768.
- [5] X. Allamigeon, P. Benchimol, S. Gaubert, M. Joswig, Log-barrier interior point methods are not strongly polynomial, *SIAM J. Appl. Algebra Geom.* 2 (2018) 140–178.
- [6] X. Allamigeon, P. Benchimol, S. Gaubert, M. Joswig, What tropical geometry tells us about the complexity of linear programming, *SIAM Rev.* 63 (2021) 123–164.
- [7] A. Altman, J. Gondzio, Regularized symmetric indefinite systems in interior point methods for linear and quadratic optimization, *Optim. Methods Softw.* 11–12 (1999) 275–302.
- [8] E.D. Andersen, K.D. Andersen, The MOSEK interior point optimizer for linear programming: an implementation of the homogeneous algorithm, in: H. Frenk, K. Roos, T. Terlaky, S. Zhang (Eds.), *High Performance Optimization*, Kluwer Academic Publishers, 2000, pp. 197–232.
- [9] E.D. Andersen, J. Gondzio, C. Mészáros, X. Xu, Implementation of interior point methods for large scale linear programming, in: T. Terlaky (Ed.), *Interior Point Methods in Mathematical Programming*, Kluwer Academic Publishers, 1996, pp. 189–252.
- [10] E.D. Andersen, C. Roos, T. Terlaky, On implementing a primal-dual interior point method for conic optimization, *Math. Program.* 95 (2003) 249–277.
- [11] B. Augustino, G. Nannicini, T. Terlaky, L. Zuluaga, Quantum interior point methods for semidefinite optimization, *Quantum* 7 (2023) 1110.
- [12] C. Barnhart, E.L. Johnson, G.L. Nemhauser, M.W.P. Savelsbergh, P.H. Vance, Branch-and-price: column generation for solving huge integer programs, *Oper. Res.* 46 (1998) 316–329.
- [13] S. Bellavia, Inexact interior point method, *J. Optim. Theory Appl.* 96 (1998) 109–121.
- [14] A. Ben-Israel, A volume associated with $m \times n$ matrices, *Linear Algebra Appl.* 167 (1992) 87–111.
- [15] J.F. Benders, Partitioning procedures for solving mixed-variables programming problems, *Numer. Math.* 4 (1962) 238–252.
- [16] M. Benzi, G. Golub, J. Liesen, Numerical solution of saddle point problems, *Acta Numer.* 14 (2005) 1–137.
- [17] L. Bergamaschi, J. Gondzio, A. Martínez, J.W. Pearson, S. Pougkakiotis, A new preconditioning approach for an interior point-proximal method of multipliers for linear and convex quadratic programming, *Numer. Linear Algebra Appl.* 28 (2021) e2361.
- [18] L. Bergamaschi, J. Gondzio, M. Venturin, G. Zilli, Inexact constraint preconditioners for linear system arising in interior point methods, *Comput. Optim. Appl.* 36 (2007) 137–147.
- [19] L. Bergamaschi, J. Gondzio, G. Zilli, Preconditioning indefinite systems in interior point methods for optimization, *Comput. Optim. Appl.* 28 (2004) 149–171.
- [20] D.P. Bertsekas, *Nonlinear Programming*, Athena Scientific, Massachusetts, 1995.
- [21] R.E. Bixby, Progress in linear programming, *ORSA J. Comput.* 6 (1994) 15–22.
- [22] S. Bocanegra, F. Campos, A. Oliveira, Using a hybrid preconditioner for solving large-scale linear systems arising from interior point methods, *Comput. Optim. Appl.* 36 (2007) 149–164.
- [23] M. Bodur, S. Dash, O. Günlük, J. Luedtke, Strengthened Benders cuts for stochastic integer programs with continuous recourse, *INFORMS J. Comput.* 29 (2017) 77–91.
- [24] J.R. Bunch, B.N. Parlett, Direct methods for solving symmetric indefinite systems of linear equations, *SIAM J. Numer. Anal.* 8 (1971) 639–655.
- [25] Bui-Trong-Lieu, P. Huard, La méthode des centres dans un espace topologique, *Numer. Math.* 8 (1966) 56–67.
- [26] S. Cafieri, M. D’Apuzzo, V. De Simone, D. di Serafino, G. Toraldo, Convergence analysis of an inexact potential reduction method for convex quadratic programming, *J. Optim. Theory Appl.* 135 (2007) 355–366.
- [27] J. Castro, A specialized interior-point algorithm for multicommodity network flows, *SIAM J. Optim.* 10 (2000) 852–877.
- [28] J. Castro, J. Cuesta, Quadratic regularizations in an interior point method for primal block-angular problems, *Math. Program.* 130 (2011) 415–445.
- [29] J. Castro, S. Nasini, F. Saldanha-da-Gama, A cutting-plane approach for large-scale capacitated multi-period facility location using a specialized interior-point method, *Math. Program.* 163 (2017) 411–444.
- [30] J. Chai, K. Toh, Preconditioning and iterative solution of symmetric indefinite linear systems arising from interior point methods for linear programming, *Comput. Optim. Appl.* 36 (2007) 221–247.
- [31] A. Chowdhury, P. London, H. Avron, P. Drineas, Faster randomized infeasible interior point methods for tall/wide linear programs, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 8704–8715.
- [32] S. Cipolla, J. Gondzio, Proximal stabilized interior point methods and low-frequency-update preconditioning techniques, *J. Optim. Theory Appl.* 197 (2023) 1061–1103.
- [33] S. Cipolla, J. Gondzio, F. Zanetti, A regularized interior point method for sparse optimal transport on graphs, *Eur. J. Oper. Res.* 319 (2024) 413–426.
- [34] D. Dadush, S. Huibert, B. Natura, L.A. Végh, A scaling-invariant algorithm for linear programming whose running time depends only on the constraint matrix, *Math. Program.* 204 (2024) 135–206.
- [35] G.B. Dantzig, *Linear Programming and Extensions*, Princeton University Press, Princeton, 1963.
- [36] G.B. Dantzig, P. Wolfe, The decomposition algorithm for linear programming, *Econometrica* 29 (1961) 767–778.
- [37] M. D’Apuzzo, V. De Simone, D. di Serafino, On mutual impact of numerical linear algebra and large-scale optimization with focus on interior point methods, *Comput. Optim. Appl.* 45 (2010) 283–310.
- [38] I. Dassios, K. Fountoulakis, J. Gondzio, A preconditioner for a primal-dual Newton conjugate gradients method for compressed sensing problems, *SIAM J. Sci. Comput.* 27 (2015) A2783–A2812.
- [39] T.A. Davies, Y. Hu, The University of Florida sparse matrix collection, *ACM Trans. Math. Softw.* 38 (2011) 1–25.
- [40] V. De Simone, D. di Serafino, J. Gondzio, S. Pougkakiotis, M. Viola, Sparse approximations with interior point methods, *SIAM Rev.* 64 (2022) 954–988.
- [41] A. Deza, T. Terlaky, Y. Zinchenko, Central path curvature and iteration-complexity for redundant Klee-Minty cubes, in: *Advances in Applied Mathematics and Global Optimization*, vol. 17, 2009, pp. 223–256.

- [42] I.I. Dikin, On the speed of an iterative process, *Upr. Sist.* 12 (1974) 54–60.
- [43] H. Dollar, N. Gould, W. Schilders, A. Wathen, Using constraint preconditioners with regularized saddle-point problems, *Comput. Optim. Appl.* 36 (2007) 249–270.
- [44] I.S. Duff, A.M. Erisman, J.K. Reid, *Direct Methods for Sparse Matrices*, Oxford University Press, New York, 1987.
- [45] C. Durazzi, V. Ruggiero, Indefinitely preconditioned conjugate gradient method for large sparse equality and inequality constrained quadratic problems, *Numer. Linear Algebra Appl.* 10 (2003) 673–688.
- [46] M. Essid, J. Solomon, Quadratically regularized optimal transport on graphs, *SIAM J. Sci. Comput.* 40 (2018) A1961–A1986.
- [47] A.V. Fiacco, G.P. McCormick, *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*, John Wiley and Sons, New York, 1968.
- [48] J.J.H. Forrest, D. Goldfarb, Steepest-edge simplex algorithms for linear programming, *Math. Program.* 57 (1992) 341–374.
- [49] A. Forsgren, P.E. Gill, M.H. Wright, Interior methods for nonlinear optimization, *SIAM Rev.* 44 (2002) 525–597.
- [50] K. Fountoulakis, J. Gondzio, P. Zhlobich, Matrix-free interior point method for compressed sensing problems, *Math. Program. Comput.* 6 (2014) 1–31.
- [51] A. Frangioni, C. Gentile, Prim-based support-graph preconditioners for min-cost flow problems, *Comput. Optim. Appl.* 36 (2007) 271–287.
- [52] A. Frangioni, S. Serra Capizzano, Spectral analysis of (sequences of) graph matrices, *SIAM J. Math. Anal.* 23 (2001) 339–348.
- [53] K.R. Frisch, The logarithmic potential method of convex programming, tech. rep., University Institute of Economics, Oslo, Norway, 1955.
- [54] R. Garfinkel, G.L. Nemhauser, *Integer Programming*, John Wiley & Sons, New York, 1972.
- [55] P.E. Gill, W. Murray, D.B. Pongceleón, M.A. Saunders, Preconditioners for indefinite systems arising in optimization, *SIAM J. Matrix Anal. Appl.* 13 (1992) 292–311.
- [56] P.E. Gill, W. Murray, M.A. Saunders, J.A. Tomlin, M.H. Wright, On the projected Newton barrier methods for linear programming and an equivalence to Karmarkar's projective method, *Math. Program.* 36 (1986) 183–209.
- [57] D. Goldfarb, M.J. Todd, Linear programming, in: G.L. Nemhauser, A.H.G. Rinnooy Kan, M.J. Todd (Eds.), *Optimization*, North Holland, Amsterdam, 1989, pp. 73–170.
- [58] J. Gondzio, Warm start of the primal-dual method applied in the cutting plane scheme, *Math. Program.* 83 (1998) 125–143.
- [59] J. Gondzio, Convergence analysis of an inexact feasible interior point method for convex quadratic programming, *SIAM J. Optim.* 23 (2013) 1510–1527.
- [60] J. Gondzio, P. González-Breis, P. Munari, New developments in the primal-dual column generation technique, *Eur. J. Oper. Res.* 224 (2013) 41–51.
- [61] J. Gondzio, P. González-Breis, P. Munari, Large-scale optimization with the primal-dual column generation method, *Math. Program. Comput.* 8 (2016) 47–82.
- [62] J. Gondzio, A. Grothey, Reoptimization with the primal-dual interior point method, *SIAM J. Optim.* 13 (2003) 842–864.
- [63] J. Gondzio, A. Grothey, Direct solution of linear systems of size 10^9 arising in optimization with interior point methods, in: R. Wyrzykowski, J. Dongarra, N. Meyer, J. Wasniewski (Eds.), *Parallel Processing and Applied Mathematics*, in: *Lecture Notes in Computer Science*, vol. 3911, Springer-Verlag, Berlin, 2006, pp. 513–525.
- [64] J. Gondzio, A. Grothey, Solving nonlinear portfolio optimization problems with the primal-dual interior point method, *Eur. J. Oper. Res.* 181 (2007) 1019–1029.
- [65] J. Gondzio, S.-M. Latva-Äijö, S.M. Siltanen, M. Lassas, F. Zanetti, Material-separating regularizer for multi-energy X-ray tomography, *Inverse Probl.* 38 (2022) 025013.
- [66] J. Gondzio, S. Pougkakiotis, J.W. Pearson, General-purpose preconditioning for regularized interior point methods, *Comput. Optim. Appl.* 83 (2022) 727–757.
- [67] J. Gondzio, R. Sarkissian, Parallel interior point solver for structured linear programs, *Math. Program.* 96 (2003) 561–584.
- [68] J. Gondzio, J.-P. Vial, Warm start and ϵ -subgradients in cutting plane scheme for block-angular linear programs, *Comput. Optim. Appl.* 14 (1999) 17–36.
- [69] C.C. Gonzaga, Path-following methods for linear programming, *SIAM Rev.* 34 (1992) 167–224.
- [70] S.A. Goreinov, I.V. Oseledets, D.V. Savostyanov, E.E. Tyrtyshnikov, N.L. Zamarashkin, How to find a good submatrix, in: *Matrix Methods: Theory, Algorithms and Applications*, World Scientific Publishers, Hackensack, NJ, 2010, pp. 247–256.
- [71] S.A. Goreinov, E.E. Tyrtyshnikov, The maximal-volume concept in approximation by low-rank matrices, in: *Structured Matrices in Mathematics, Computer Science, and Engineering*, I, Boulder, CO, 1999, in: *Contemporary Mathematics*, vol. 280, American Mathematical Society, Providence, RI, 2001, pp. 47–51.
- [72] O. Güler, D. den Hertog, C. Roos, T. Terlaky, T. Tsuchiya, Degeneracy in interior point methods for linear programming: a survey, *Ann. Oper. Res.* 46 (1993) 107–138.
- [73] J.A.J. Hall, Towards a practical parallelisation of the simplex method, *Comput. Manag. Sci.* 7 (2010) 139–170.
- [74] J.A.J. Hall, K.I.M. McKinnon, Hyper-sparsity in the revised simplex method and how to exploit it, *Comput. Optim. Appl.* 32 (2005) 259–283.
- [75] L. Hall, R.J. Vanderbei, Two-thirds is sharp for affine scaling, *Oper. Res. Lett.* 13 (1993) 197–201.
- [76] Huard, Resolution of mathematical programming with nonlinear constraints by the method of centers, in: J. Abadie (Ed.), *Nonlinear Programming*, North Holland, Amsterdam, 1967, pp. 207–219.
- [77] IBM, *IBM ILOG CPLEX*, 2019.
- [78] J.J. Júdice, J. Patricio, L.F. Portugal, M.G.C. Resende, G. Veiga, A study of preconditioners for network interior point methods, *Comput. Optim. Appl.* 24 (2003) 5–35.
- [79] P. Kall, S.W. Wallace, *Stochastic Programming*, John Wiley & Sons, Chichester, 1994.
- [80] L.V. Kantorovich, On the translocation of masses, *Manag. Sci.* 5 (1958) 1–4.
- [81] L.V. Kantorovich, *The Best Use of Economic Resources*, Harvard University Press, Harvard, 1965.
- [82] N.K. Karmarkar, A new polynomial-time algorithm for linear programming, *Combinatorica* 4 (1984) 373–395.
- [83] C. Keller, N.I.M. Gould, A.J. Wathen, Constraint preconditioning for indefinite linear systems, *SIAM J. Matrix Anal. Appl.* 21 (2000) 1300–1317.
- [84] C.T. Kelley, *Iterative Methods for Linear and Nonlinear Equations*, *Frontiers in Applied Mathematics*, vol. 16, SIAM, Philadelphia, 1995.
- [85] I. Kerenidis, A. Prakash, D. Szilágyi, Quantum algorithms for second-order cone programming and support vector machines, *Quantum* 5 (2021) 427.
- [86] L.G. Khachiyan, A polynomial algorithm in linear programming, *Sov. Math. Dokl.* 20 (1979) 191–194.
- [87] V. Klee, G. Minty, How good is the simplex algorithm, in: O. Shisha (Ed.), *Inequalities-III*, Academic Press, 1972, pp. 159–175.
- [88] M. Kojima, N. Megiddo, S. Mizuno, A primal-dual infeasible-interior-point algorithm for linear programming, *Math. Program.* 61 (1993) 263–280.
- [89] M. Kojima, S. Mizuno, A. Yoshise, A primal-dual interior point algorithm for linear programming, in: N. Megiddo (Ed.), *Progress in Mathematical Programming: Interior-Point Algorithms and Related Methods*, Springer Verlag, Berlin, Germany, 1989, pp. 29–47.
- [90] P. Kovacs, Minimum-cost flow algorithms: an experimental evaluation, *Optim. Methods Softw.* 30 (2015) 94–127.
- [91] Y. Lee, A. Sidford, Solving linear programs with $\tilde{O}(\sqrt{\text{rank}})$ linear system solves, tech. rep., Stanford University, 2013.
- [92] Y. Lee, A. Sidford, Path finding methods for linear programming: solving linear programs in $\tilde{O}(\sqrt{\text{rank}})$ iterations and faster algorithms for maximum flow, in: *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, 2014, pp. 424–433.
- [93] Y. Lee, A. Sidford, Efficient inverse maintenance and faster algorithms for linear programming, in: *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, 2015, pp. 230–249.
- [94] D. Lewis, Finite dimensional subspaces of l_p , *Stud. Math.* 63 (1978) 207–212.
- [95] M. Lobo, L. Vandenbergh, S. Boyd, H. Lebret, Applications of second-order cone programming, *Linear Algebra Appl.* 284 (1998) 193–228.
- [96] M.E. Lübbecke, J. Desrosiers, Selected topics in column generation, *Oper. Res.* 53 (2005) 1007–1023.
- [97] I.J. Lustig, R.E. Marsten, D.F. Shanno, On implementing Mehrotra's predictor-corrector interior point method for linear programming, *SIAM J. Optim.* 2 (1992) 435–449.
- [98] I.J. Lustig, R.E. Marsten, D.F. Shanno, Interior point methods for linear programming: computational state of the art, *ORSA J. Comput.* 6 (1994) 1–14.

- [99] I. Maros, Computational Techniques of the Simplex Method, 1st ed., Kluwer Academic Publishers, Boston, 2003.
- [100] N. Megiddo, Pathways to the optimal set in linear programming, in: N. Megiddo (Ed.), Progress in Mathematical Programming: Interior-Point Algorithms and Related Methods, Springer Verlag, Berlin, Germany, 1989.
- [101] N. Megiddo, On finding primal- and dual-optimal bases, ORSA J. Comput. 3 (1991) 63–65.
- [102] S. Mehrotra, On the implementation of a primal–dual interior point method, SIAM J. Optim. 2 (1992) 575–601.
- [103] C. Mészáros, Sparsity in convex quadratic programming with interior point methods, Optim. Methods Softw. 21 (2006) 733–745.
- [104] C. Mészáros, Detecting “dense” columns in interior point methods for linear programming, Comput. Optim. Appl. 36 (2007) 309–320.
- [105] J.E. Mitchell, Karmarkar’s Algorithm and Combinatorial Optimization Problems, PhD thesis, Cornell University, 1988.
- [106] J.E. Mitchell, B. Borchers, Solving real-world linear ordering problems using a primal-dual interior point cutting plane method, Ann. Oper. Res. 62 (1996) 253–276.
- [107] J.E. Mitchell, M.J. Todd, Solving combinatorial optimization problems using Karmarkar’s algorithm, Math. Program. 56 (1992) 245–284.
- [108] S. Mizuno, F. Jarre, Global and polynomial-time convergence of an infeasible-interior point algorithm using inexact computation, Math. Program. 84 (1999) 105–122.
- [109] M. Mohammadisiahroudi, R. Fakhimi, T. Terlaky, Efficient use of quantum linear system algorithms in interior point methods for linear optimization, tech. rep., Lehigh University, February 2023, <https://doi.org/10.48550/arXiv.2205.01220>.
- [110] M. Mohammadisiahroudi, R. Fakhimi, Z. Wu, T. Terlaky, An inexact feasible interior point method for linear optimization with high adaptability to quantum computers, tech. rep., Lehigh University, July 2023, <https://doi.org/10.48550/arXiv.2307.14445>.
- [111] R.D.C. Monteiro, I. Adler, M.G.C. Resende, A polynomial-time primal-dual affine scaling algorithm for linear and convex quadratic programming and its power series extension, Math. Oper. Res. 15 (1990) 191–214.
- [112] P. Munari, J. Gondzio, Using the primal-dual interior point algorithm within the branch-price-and-cut method, Comput. Oper. Res. 40 (2013) 2026–2036.
- [113] B. Murtagh, M. Saunders, Large-scale linearly constrained optimization, Math. Program. 14 (1978) 41–72.
- [114] G.L. Nemhauser, L.A. Wolsey, Integer and Combinatorial Optimization, John Wiley & Sons, New York, 1988.
- [115] Y. Nesterov, Towards non-symmetric conic optimization, Optim. Methods Softw. 27 (2012) 893–917.
- [116] Y. Nesterov, A. Nemirovski, Interior Point Polynomial Algorithms in Convex Programming: Theory and Applications, SIAM, Philadelphia, 1994.
- [117] Y. Nesterov, M.J. Todd, Primal-dual interior-point methods for self-scaled cones, SIAM J. Optim. 8 (1998) 324–364.
- [118] A.R.L. Oliveira, A New Class of Preconditioners for Large-Scale Linear Systems from Interior Point Methods for Linear Programming, PhD thesis, Department of Computational and Applied Mathematics, Rice University, Houston, TX 77005, 1997.
- [119] A.R.L. Oliveira, D.C. Sorensen, A new class of preconditioners for large-scale linear systems from interior point methods for linear programming, Linear Algebra Appl. 394 (2005) 1–24.
- [120] D. Papp, S. Yildiz, On “A homogeneous interior-point algorithm for non-symmetric convex conic optimization”, tech. rep., University of North Carolina, June 2018, <https://doi.org/10.48550/arXiv.1712.00492>.
- [121] G. Peyre, M. Cuturi, Computational optimal transport: with applications to data science, Found. Trends Mach. Learn. 11 (2019) 355–607.
- [122] S. Pougkakiotis, J. Gondzio, Dynamic non-diagonal regularization in interior point methods for linear and convex quadratic programming, J. Optim. Theory Appl. 181 (2019) 905–945.
- [123] J. Renegar, Incorporating condition measures into the complexity theory of linear programming, SIAM J. Optim. 5 (1995) 506–524.
- [124] M.G.C. Resende, G. Veiga, An implementation of the dual affine scaling algorithm for minimum cost flow on bipartite uncapacitated networks, SIAM J. Optim. 3 (1993) 516–537.
- [125] M. Saunders, Cholesky-based methods for sparse least squares: the benefits of regularization, in: L. Adams, L. Nazareth (Eds.), Linear and Nonlinear Conjugate Gradient-Related Methods, SIAM, Philadelphia, USA, 1996, pp. 92–100.
- [126] O. Schenk, A. Wächter, M. Weiser, Inertia revealing preconditioning for large-scale nonconvex constrained optimization, SIAM J. Sci. Comput. 31 (2008) 939–960.
- [127] L. Schork, J. Gondzio, Implementation of an interior point method with basis preconditioning, Math. Program. Comput. 12 (2020) 603–635.
- [128] L. Schork, J. Gondzio, Rank revealing Gaussian elimination by the maximum volume concept, Linear Algebra Appl. 592 (2020) 1–19.
- [129] D.F. Shanno, R.J. Vanderbei, Interior-point methods for nonconvex nonlinear programming: orderings and higher-order methods, Math. Program. 87 (2000) 303–316.
- [130] A. Skajaa, Y. Ye, A homogeneous interior-point algorithm for nonsymmetric convex conic optimization, Math. Program. 150 (2015) 391–422.
- [131] G.W. Stewart, On scaled projections and pseudoinverses, Linear Algebra Appl. 112 (1989) 189–193.
- [132] U.H. Suhl, L.M. Suhl, Computing sparse LU factorizations for large-scale linear programming bases, ORSA J. Comput. 2 (1990) 325–335.
- [133] T. Terlaky, Criss-cross pivoting rules, in: C. Floudas, P. Pardalos (Eds.), Encyclopedia of Optimization, Kluwer Academic Publishers, 2009, pp. 584–590.
- [134] M. Todd, Semidefinite optimization, Acta Numer. 10 (2001) 515–560.
- [135] T. Tsuchiya, Global convergence of the affine scaling methods for degenerate linear programming problems, Math. Program. 52 (1991) 377–404.
- [136] L. Vandenbergh, S. Boyd, Semidefinite programming, SIAM Rev. 38 (1996) 49–95.
- [137] F. Vanderbeck, On Dantzig-Wolfe decomposition in integer programming and ways to perform branching in a branch-and-price algorithm, Oper. Res. 48 (2000) 111–128.
- [138] R.J. Vanderbei, Symmetric quasidfinite matrices, SIAM J. Optim. 5 (1995) 100–113.
- [139] R.J. Vanderbei, LOQO: an interior point code for quadratic programming, Optim. Methods Softw. 11–12 (1999) 451–484.
- [140] R.J. Vanderbei, D.F. Shanno, An interior-point algorithm for nonconvex nonlinear programming, Comput. Optim. Appl. 13 (1999) 231–252.
- [141] S.A. Vavasis, Y. Ye, A primal-dual interior point method whose running time depends only on the constraint matrix, Math. Program. 74 (1996) 79–120.
- [142] N.K. Vishnoi, $Lx = b$ Laplacian solvers and their algorithmic applications, Found. Trends Theor. Comput. Sci. 8 (2013) 1–141.
- [143] A. Wächter, L. Biegler, On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming, Math. Program. 106 (2006) 25–57.
- [144] S.J. Wright, Primal-Dual Interior-Point Methods, SIAM, Philadelphia, 1997.
- [145] Z. Wu, M. Mohammadisiahroudi, B. Augustino, X. Yang, T. Terlaky, An inexact feasible quantum interior point method for linearly constrained quadratic optimization, tech. rep., Lehigh University, January 2023, <https://doi.org/10.48550/arXiv.2301.05357>.
- [146] H. Yamashita, A globally convergent primal-dual interior point method for constrained optimization, Optim. Methods Softw. 10 (1998) 443–469.
- [147] H. Yamashita, H. Yabe, Superlinear and quadratic convergence of some primal-dual interior point methods for constrained optimization, Math. Program. 75 (1996) 377–397.
- [148] E.A. Yildirim, M.J. Todd, An interior-point approach to sensitivity analysis in degenerate linear programs, SIAM J. Optim. 12 (2002) 692–714.
- [149] E.A. Yildirim, S.J. Wright, Warm-start strategies in interior-point methods for linear programming, SIAM J. Optim. 12 (2002) 782–810.
- [150] F. Zanetti, J. Gondzio, An interior-point-inspired algorithm for linear programs arising in discrete optimal transport, INFORMS J. Comput. 35 (2023) 1061–1078.
- [151] F. Zanetti, J. Gondzio, A new stopping criterion for Krylov solvers applied in interior point methods, SIAM J. Sci. Comput. 45 (2023) A703–A728.
- [152] Y. Zhang, On the convergence of a class of infeasible interior-point methods for the horizontal linear complementarity problem, SIAM J. Optim. 4 (1994) 208–227.